



Application Development Internship

Project Report on

OuroMini: Wireless Security Testing Platform

Submitted by

K. Rupa Yeshvitha 2311CS040082

K. Jeshwanth 2311CS040072

K. Sathya Sai 2311CS040088

K. Vamshi 2311CS040078

B. Tech III Year I Semester

Under the Guidance of

Dr. G. Latha (Associate Professor)

Department of CSE (Cyber Security)

Malla Reddy University, Hyderabad

November 2025



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

CERTIFICATE

This is to certify that, the Application Development Internship Project entitled “OUROMINI - ESP32-WROOM-32 BASED WIRELESS SECURITY TESTING AND MONITORING PLATFORM”, has been successfully completed and submitted by **K. Rupa Yeshvitha (2311CS040082), K. Sathya Sai (2311CS040088), K. Jeshwanth (2311CS040072), K. Vamshi (2311CS040078)**, B.Tech III year I semester, Department of CSE (CS) during November 2025. The results embodied in this report have not been submitted earlier to any other university or institute for any other internship or the award of any degree or diploma.

Internal Guide

Dr. G. Latha

(Associate Professor)

Dean CS, IoT & ECE

Dr. G. Anand Kumar

(Professor)

External Examiner

ACKNOWLEDGEMENTS

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude to **Dr. V. S. K Reddy, Vice-Chancellor**, for his visionary leadership and unwavering commitment to academic excellence.

We would also like to express our deepest gratitude to our project guide **Dr. G. Latha Associate Professor, Dept. of Cyber Security & IoT**, whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout this project for successful outcomes.

We are also grateful to **Dr. G. Anand Kumar, Dean, Cyber Security & IoT Dept.**, for initiating and providing us with the necessary resources and facilities to carry out this project.

We extend our gratitude to our **AD – PRC Convener & Coordinator, Dr. G. Latha**, for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

K. RUPA YESHVITHA	2311CS040082
K. SATHYA SAI	2311CS040088
K. JESHWANTH	2311CS040072
K. VAMSHI	2311CS040078

ABSTRACT

The modern digital ecosystem is increasingly dependent on wireless communication technologies, which form the backbone of contemporary networking infrastructures. As connectivity grows ubiquitous through Wi-Fi, Bluetooth, and emerging IoT protocols, so too do the associated cybersecurity risks. The continuous proliferation of wireless devices has opened new vectors for unauthorized access, data interception, and signal manipulation. Despite the critical need for robust wireless security assessment, existing solutions remain fragmented, complex, and prohibitively expensive for widespread academic and research adoption. Tools like software-defined radios or professional network analyzers require high computational resources and domain expertise, creating a technological divide between theoretical cybersecurity education and practical implementation.

To bridge this gap, OuroMini has been conceived as an embedded, low-cost, and multifunctional wireless security testing and monitoring platform based on the ESP32-WROOM-32 microcontroller. This compact yet powerful system unifies multiple wireless testing functionalities within a single board, providing capabilities such as access point (AP) scanning, probe and PMKID sniffing, client mapping, and channel-based spectrum activity analysis. By leveraging the dual-core architecture and built-in Wi-Fi capabilities of ESP32, OuroMini enables real-time packet monitoring, metadata extraction, and analysis without reliance on external hardware or display modules. The device interacts with users through a serial command interface, making it fully compatible with cross-platform environments and minimal host resources.

The project emphasizes modular firmware design, allowing seamless integration of additional wireless functionalities such as BLE and RFID in future revisions. Each functional module—ranging from the initialization and Wi-Fi scan modules to the probe and PMKID sniffers—operates as an independent logical component within a FreeRTOS-based firmware ecosystem. The result is a system that efficiently balances performance, power consumption, and real-time responsiveness.

From an educational and research standpoint, OuroMini represents a pivotal step toward democratizing cybersecurity experimentation. Its open-source nature, combined with portability and low production cost, makes it suitable for classroom demonstrations, security workshops, and professional penetration testing toolkits alike. Furthermore, its serial data format can be extended to generate visual analytics, heat maps, and pictographic representations of network behavior for deeper insight.

Ultimately, OuroMini serves as a bridge between embedded system engineering and cybersecurity research. It redefines how wireless security testing tools can be designed—compact, affordable, and extensible—while remaining fully aligned with ethical and educational objectives. By consolidating scanning, sniffing, and analysis within a single embedded device, it empowers users to understand, visualize, and strengthen wireless security ecosystems in real time.

TABLE OF CONTENTS

S.NO	Contents	Page no
1	Introduction	1-2
	1.1 Problem Definition & Description	1
	1.2 Objective of the Project	1
	1.3 Scope of the Project	2
2	System Analysis	3-4
	2.1 Existing System	3
	2.1.1 Background & Literature Survey	3
	2.1.2 Limitations of Existing System	4
	2.2 Proposed System	5
	2.2.1 Advantages of Proposed System	6
	2.3 Software & Hardware Requirements	7-8
	2.3.1 Software Requirements	7
	2.3.2 Hardware Requirements	8
	2.4 Feasibility Study	9-10
	2.4.1 Technical Feasibility	9
	2.4.2 Robustness & Reliability	9
	2.4.3 Economic Feasibility	10
3	Architectural Design	11-29
	3.1 Modules Design	11-19
	3.2 Method & Algorithm Design	19
	3.2.1 Method	19-20
	3.2.2 Algorithms	20-23
	3.3 Project Architecture	24-29
	3.3.1 Architecture	25

	3.3.2 Dataflow Diagram	26-27
	3.3.3 Class Diagram	28
	3.3.4 Use case Diagram	29
	3.3.5 Sequence Diagram	29
	3.3.6 Activity Diagram	29
4	Implementation	30-49
	4.1 Coding Blocks	30-31
	4.2 Sample code	32-40
	4.3 Testing	41
5	Results	42-54
6	Conclusion	46
7	Future Scope	47
	References	48
	Appendix	49

1. INTRODUCTION

1.1 Problem Definition and Description

In the modern era of ubiquitous wireless communication, network infrastructures have become the lifelines of digital ecosystems. Wi-Fi, Bluetooth, and emerging IoT protocols enable seamless connectivity between devices, users, and services. However, this rapid technological advancement has simultaneously widened the landscape of security threats. Unauthorized access points, rogue devices, packet spoofing, and signal interception are now common vulnerabilities affecting both domestic and enterprise environments.

Traditional approaches to wireless auditing and cybersecurity analysis rely heavily on resource-intensive software and specialized hardware such as SDRs (Software Defined Radios), multiple network adapters, and complex PC-based configurations. These setups, although effective, are expensive, difficult to operate, and unsuitable for field demonstrations or embedded learning. As a result, cybersecurity education and research often face practical limitations when attempting to bridge theoretical understanding with real-world experimentation.

To overcome these challenges, the **OuroMini** project introduces a compact, portable, and low-cost wireless monitoring and analysis system built entirely on the **ESP32-WROOM-32** microcontroller. This embedded solution eliminates dependence on bulky external hardware and software layers, enabling users to scan wireless networks, sniff packets, and analyze network activity directly through a serial or terminal interface. The design emphasizes modular firmware that allows continuous operation in multiple modes—such as access-point scanning, beacon/probe sniffing, PMKID capture, and client mapping—while maintaining minimal power consumption. OuroMini serves as an innovative research platform for students, penetration testers, and educators, allowing hands-on exploration of wireless protocol behavior, network security mechanisms, and data-driven signal analytics.

1.2 Objective of the Project

The primary objective of OuroMini is to design and develop an **embedded wireless security testing and monitoring platform** that is cost-efficient, easily deployable, and educationally impactful. The specific goals include:

- **To integrate multiple wireless analysis functions**—including access-point discovery, signal strength measurement, PMKID sniffing, and packet monitoring—within a single ESP32-based module.
- **To eliminate dependence on high-end computing devices** by enabling standalone real-time analysis through serial communication.
- **To create an extensible and modular firmware structure** that can be expanded with additional protocols such as BLE, Sub-GHz, or TinyML-based anomaly detection.
- **To support cybersecurity education and training** by providing a hands-on, safe, and open-source tool for studying wireless vulnerabilities, authentication behaviors, and data flow patterns.
- **To promote affordability and accessibility** in wireless research by offering a solution that can be replicated by academic institutions, hobbyists, and professional testers alike.

Ultimately, OuroMini aims to democratize the process of wireless cybersecurity exploration—making advanced scanning, sniffing, and network behavior analysis tools available in a small, energy-efficient, and open-hardware format.

1.3 Scope of the Project

The scope of OuroMini encompasses the complete development of a **multi-functional, ESP32-based wireless testing system** that integrates both hardware and software layers in an optimized embedded framework. It extends across the following dimensions:

- **Educational Use:** As a hands-on platform for cybersecurity students to learn about Wi-Fi packet structures, AP/STA interactions, and wireless scanning logic without needing a PC-dependent setup.
- **Research Application:** Providing a testbed for lightweight intrusion detection, protocol behavior modeling, and wireless signal pattern visualization.
- **Field Operations:** Serving as a portable tool for professionals to perform on-site network reconnaissance, audit wireless strength, or detect unauthorized access points.
- **Extension and Customization:** Its modular architecture allows integration of complementary functions such as BLE sniffing, NFC exploration, or signal triangulation in future firmware updates.

Unlike traditional analyzers, OuroMini has **no display dependency**, relying instead on **serial or web terminal visualization**. This design ensures flexibility, minimal power draw, and universal compatibility across operating systems. The simplicity of deployment and open-source firmware makes OuroMini an adaptable foundation for classroom instruction, ethical hacking workshops, or rapid-prototyping scenarios.

2.1 Existing System

2.1.1 Background & Literature Survey

Before the development of compact, embedded wireless monitoring systems like **OuroMini**, most network reconnaissance and security analysis tools were limited to computer-based environments requiring dedicated adapters and complex setup procedures. Traditional Wi-Fi auditing systems—such as **Aircrack-ng**, **Wireshark**, or **Kismet**—though powerful, depend on high-performance processors, multiple wireless chipsets in monitor mode, and Linux-based configurations. These requirements make them impractical for lightweight field deployment or real-time embedded experimentation. Moreover, such setups often consume high power, rely on external storage, and lack the immediacy and portability demanded by researchers or students engaging in practical cybersecurity testing.

Over the years, significant advancements have been made in the domain of **wireless network security** and **embedded IoT systems**. Researchers have explored methods for passive Wi-Fi scanning, packet sniffing, and device fingerprinting using general-purpose microcontrollers and single-board computers such as the **Raspberry Pi**, **ESP8266**, and **ESP32**. Projects like the *ESP8266 Deauther* and *ESP32 Marauder* introduced low-cost methods for packet analysis, network scanning, and basic attack simulation. These systems demonstrated the feasibility of embedding wireless monitoring capabilities into microcontrollers, drastically reducing cost and increasing accessibility. However, many of these early systems suffered from fragmented firmware architectures, lack of modular scalability, and inconsistent performance across network environments.

Academic literature on IoT and cybersecurity education has emphasized the need for **portable, affordable, and safe-to-use wireless research platforms** that can be integrated into teaching laboratories without regulatory risk. Studies from IEEE and ACM highlight that introducing network reconnaissance concepts through embedded tools fosters deeper understanding of real-world attack surfaces and defense mechanisms. Despite this, most commercially available devices remain either too expensive or too restricted for open experimentation.

In parallel, the growing reliance on **Wi-Fi-enabled IoT devices** has exposed new vulnerabilities in everyday environments. Research in wireless security testing underscores the necessity of compact field analyzers that can detect rogue access points, analyze channel occupancy, and monitor packet flow anomalies. Yet, existing solutions are primarily software-bound or tethered to full operating systems, creating barriers for mobility and real-time response.

These limitations have collectively inspired the development of **OuroMini**, a self-contained, firmware-driven wireless monitoring and security analysis device. Built entirely on the **ESP32-WROOM-32**, OuroMini consolidates scanning, sniffing, and network interaction functions into a single embedded framework without requiring a PC or external adapter. By leveraging the dual-core capabilities of the ESP32 and its integrated Wi-Fi transceiver, OuroMini overcomes the limitations of traditional software-defined tools—delivering real-time network intelligence through a minimalistic and efficient command interface.

OuroMini thus represents a new generation of embedded cybersecurity systems that merge **academic utility, technical sophistication, and operational independence**. It bridges the gap between theoretical network security models and hands-on experimentation, providing a scalable foundation for future IoT-oriented intrusion detection, wireless analytics, and embedded AI-driven defense research.

2.1.2 Limitations of Existing System

Existing wireless monitoring and network analysis systems exhibit several key limitations that hinder their accessibility, adaptability, and real-time performance in field environments. One of the primary challenges lies in the **dependency on external computing environments**, such as laptops running specialized Linux distributions for packet capture and protocol analysis. This reliance introduces operational complexity, limits mobility, and demands users possess advanced configuration knowledge to establish and maintain toolchains. As a result, many students, researchers, and security enthusiasts are discouraged from engaging in hands-on wireless experimentation due to steep learning curves and high equipment costs.

Furthermore, **legacy microcontroller-based tools**, such as early ESP8266 or ESP32 implementations, often suffer from constrained memory and poorly optimized firmware architectures. Many of these systems cannot maintain stable multi-channel scanning or perform concurrent sniffing and analysis operations without system crashes or dropped packets. In addition, their interfaces—frequently limited to serial outputs or basic OLED displays—lack meaningful visualization or structured feedback, making real-time interpretation of captured data difficult.

Another critical limitation of existing solutions is their **lack of modularity and unified design**. Most available tools operate as single-function utilities, capable of either scanning access points, capturing probe requests, or detecting packets—but rarely integrating these features cohesively. This fragmentation reduces their effectiveness in comparative network testing or educational demonstrations where multiple modes of operation are essential. Likewise, the absence of integrated data logging and cross-protocol interaction (e.g., Wi-Fi, BLE, sub-GHz) restricts the system’s adaptability for broader IoT security studies.

Power efficiency and heat management also remain overlooked aspects in many designs. Conventional devices frequently consume more power than their embedded supplies can sustain, especially during high-frequency channel sweeps or active deauthentication simulations. Combined with the absence of optimized firmware scheduling, this leads to reduced reliability, limited runtime, and eventual hardware instability.

Finally, most previous systems fail to offer a **consistent, user-friendly command-line interface (CLI)** or structured command management for multi-mode operations. This makes testing and debugging less intuitive, especially for users performing rapid data collection or switching between scanning modes.

These deficiencies collectively emphasize the need for a new generation of embedded wireless research tools that are **compact, efficient, modular, and autonomous**. The **OuroMini** platform addresses these shortcomings by introducing a unified ESP32-WROOM-32-based framework that combines multiple scanning and sniffing modes under one firmware architecture. It operates without the need for a host

computer, optimizes resource scheduling for stability, and employs a flexible CLI for dynamic control. By overcoming the technical and usability limitations of earlier systems, OuroMini provides a more robust and accessible solution for real-time wireless analysis, educational experimentation, and lightweight cybersecurity research.

2.2 Proposed System

The **OuroMini Wireless Security and Monitoring Platform** addresses the limitations of traditional wireless analysis systems by integrating modern embedded hardware with optimized firmware for real-time network reconnaissance, data acquisition, and security testing. It is built around the **ESP32-WROOM-32** microcontroller, a dual-core SoC that offers both high processing capability and integrated Wi-Fi/Bluetooth functionality, enabling simultaneous multi-channel scanning and protocol sniffing without the need for external computing devices. OuroMini unifies key functionalities such as **access point (AP) scanning, beacon and probe sniffing, PMKID/EAPOL capture, and station tracking** into a single embedded firmware environment, accessible through a structured **command-line interface (CLI)** for efficient user interaction.

Unlike conventional systems that depend heavily on PCs or Linux-based toolchains, OuroMini operates **completely standalone**, outputting structured data directly via serial communication or web-based visualization modules. This allows users to analyze nearby wireless networks, detect device probes, and monitor real-time signal strength (RSSI) variations directly from the ESP32 itself. The firmware is carefully optimized to manage Wi-Fi channel hopping, asynchronous packet handling, and adaptive timing control to ensure minimal packet loss and stable continuous operation.

The system architecture is modular, allowing integration of future add-ons such as **BLE scanning, Sub-GHz receivers, or TinyML-based anomaly detection**, thus providing a scalable base for advanced research and development. Its power-efficient design enables long operational runtimes using a **single lithium-ion battery**, with optional onboard voltage monitoring to ensure hardware protection during extended field use.

OuroMini's functionality extends beyond typical scanning utilities by offering **multi-layer sniffing and adaptive monitoring capabilities**, allowing it to capture beacon frames, association requests, and EAPOL packets in structured formats suitable for both human-readable output and data-driven analysis. Its architecture supports dual operational modes — **passive reconnaissance** (for safe, non-intrusive scanning) and **active interaction** (for protocol testing and controlled packet injection in research scenarios).

By combining ESP32's robust wireless stack with carefully engineered firmware logic, OuroMini demonstrates how **a low-cost, open, and portable device can perform complex network reconnaissance tasks traditionally reserved for high-end setups**. The project serves as both a **research and educational platform**, providing cybersecurity practitioners, students, and developers with a tangible tool for studying wireless behavior, understanding security mechanisms, and conducting lawful penetration testing in isolated environments.

In essence, **OuroMini** encapsulates the principles of **embedded intelligence, efficiency, and autonomy** — redefining how portable devices can participate in wireless network exploration and cybersecurity experimentation. It represents the convergence of **IoT hardware design, wireless protocol engineering, and cybersecurity analytics** within a single cohesive system, built to be open, extensible, and aligned with responsible security research practices.

2.2.1 Advantages of Proposed System

The proposed **OuroMini** system introduces several key advantages that distinguish it from conventional wireless monitoring and analysis tools. Foremost among these is its **complete standalone operation**, made possible by the integration of the **ESP32-WROOM-32** microcontroller, which consolidates Wi-Fi communication, data capture, and command processing within a single embedded platform. This eliminates the dependency on external computers, Linux toolchains, or specialized adapters, significantly improving portability, efficiency, and ease of deployment in field environments.

A major benefit of OuroMini is its **modular and unified firmware architecture**, which combines multiple functionalities—such as AP scanning, beacon sniffing, PMKID capture, and station mapping—under one coherent interface. This integrated design simplifies workflow management, allowing researchers and cybersecurity practitioners to conduct multi-layered wireless analysis seamlessly without switching tools or reconfiguring setups. The inclusion of a structured **command-line interface (CLI)** provides direct, low-latency control over system operations, making data collection and testing both intuitive and reliable.

Another notable advantage is OuroMini's **real-time packet analysis capability**, which enables the system to capture and interpret network traffic dynamically. It provides contextual insights such as **signal strength (RSSI)**, **channel utilization**, and **access point-client relationships**, allowing users to study wireless behavior in realistic conditions. Its **optimized channel-hopping algorithm** ensures continuous scanning across multiple frequencies with minimal packet loss, providing higher temporal accuracy than many comparable microcontroller-based solutions.

The device's **energy-efficient and lightweight design** makes it ideal for long-duration fieldwork and classroom demonstrations. Powered by a lithium-ion source, it incorporates efficient power management strategies to maintain stable voltage levels during active scans. Moreover, the minimal hardware footprint reduces electromagnetic interference and ensures consistent wireless reception.

In addition, OuroMini's **open and extensible framework** allows for future upgrades such as BLE, Sub-GHz communication, or TinyML integration. This flexibility supports evolving cybersecurity research needs and educational applications without requiring a hardware redesign. The system's modular codebase promotes collaborative development, making it adaptable to emerging wireless standards or experimental firmware extensions.

Overall, OuroMini presents a **smarter, safer, and more efficient** alternative to existing tools. Its combination of autonomous operation, advanced data handling, and optimized embedded architecture positions it as a robust educational and professional platform for **real-time wireless reconnaissance**, **IoT security analysis**, and **network behavior visualization**.

2.3 Software & Hardware Requirements

2.3.1 Software Requirements

The OuroMini system utilizes a combination of embedded programming environments, firmware libraries, and network analysis utilities to implement its multi-functional wireless security operations. Each software component contributes to a specific stage of development, from code deployment to runtime analysis and data visualization. The key software requirements are as follows:

- **Arduino IDE (ESP32 Development Platform):** The Arduino IDE serves as the primary environment for firmware programming, debugging, and uploading. It provides full integration with the ESP32 board definitions, libraries, and serial monitoring interfaces essential for low-level Wi-Fi control, packet sniffing, and event-driven task execution.
- **ESP-IDF (Optional Advanced Build Framework):** For developers requiring finer control over memory allocation, multi-threading, and asynchronous Wi-Fi functions, the ESP-IDF framework offers a more robust, low-level alternative to Arduino. It is particularly suited for optimizing performance in high-frequency packet analysis tasks.
- **Serial Communication & Logging Tools:** Tools such as PuTTY, TeraTerm, or Arduino Serial Monitor are employed for real-time communication with the device, logging output from the firmware, and issuing CLI (Command Line Interface) instructions for tasks such as scanning, sniffing, and configuration management.
- **Python-Based Data Visualization Utilities (Optional):** Post-capture data can be analyzed or visualized through Python scripts that plot RSSI variations, channel activity, or device presence graphs. These scripts facilitate post-processing and academic interpretation of network scan data.
- **Firmware Libraries:** Essential libraries include *WiFi.h*, *esp_wifi.h*, *EEPROM.h*, and *SPIFFS.h*, which enable wireless functionality, configuration storage, and asynchronous scanning. Additional support modules handle memory optimization, timing, and logging mechanisms.

2.3.2 Hardware Requirements

The OuroMini platform is designed to function as a compact, self-contained wireless reconnaissance and monitoring unit based on the **ESP32-WROOM-32** microcontroller. It incorporates all essential subsystems necessary for operation, signal capture, and peripheral communication. The major hardware components are listed below:

- **ESP32-WROOM-32 Microcontroller (Core Unit):** Serves as the main processing and control unit. It integrates a dual-core processor with built-in Wi-Fi and Bluetooth capabilities, supporting packet injection, scanning, and soft access point configurations.
- **Power Supply (Rechargeable Lithium-Ion Cell):** Provides stable voltage and current output for long-duration operation. Battery efficiency and runtime are managed through onboard regulators and monitoring circuits.
- **Antenna and RF Front-End (Integrated PCB Antenna):** Enables effective 2.4 GHz signal capture for Wi-Fi monitoring and sniffing operations. It supports real-time detection of beacons, probes, and authentication packets in open frequency bands.
- **Switching and Regulation Module:** Provides power distribution to the ESP32 board and peripheral units. Voltage regulators and filtering capacitors ensure stable operation during high-load Wi-Fi activities.
- **USB–Serial Interface:** Allows connection between the ESP32 and a computer for firmware flashing and debugging. It supports 115200 baud communication with serial debugging tools.
- **Status Indicators (LEDs):** Simple indicators used for operational feedback, such as power status, scanning activity, or error states. These indicators assist during firmware testing and live demonstrations.
- **Prototype Board or Perfboard Assembly:** Provides the structural foundation for soldered connections, ensuring electrical integrity and mechanical stability of the assembled circuit.
- **Enclosure/Chassis:** A lightweight acrylic or 3D-printed body houses the ESP32 and associated components, protecting the circuitry from environmental interference and providing an organized interface layout for testing.

2.4 Feasibility Study

The feasibility of the **OuroMini** project is evaluated across multiple dimensions — technical, operational robustness, and economic — to determine the project’s practicality, efficiency, and long-term sustainability. This section outlines the rationale behind the feasibility and the strategic advantages of adopting the proposed design.

2.4.1 Technical Feasibility

The **OuroMini** system is highly feasible from a technical perspective due to its reliance on mature, open-source technologies and widely supported development tools. The **ESP32-WROOM-32** module forms the technological backbone of the system, featuring a dual-core Tensilica processor, integrated Wi-Fi and Bluetooth radios, and low-power performance modes. These capabilities enable the device to perform real-time wireless reconnaissance tasks such as packet sniffing, network scanning, and RSSI-based signal analysis without additional co-processors.

The development workflow is supported by a strong software ecosystem, including the **Arduino IDE** and **ESP-IDF** frameworks, which provide comprehensive library support for Wi-Fi management, asynchronous scanning, and serial debugging. The technical feasibility is further reinforced by the compatibility of the ESP32 with external peripherals such as lithium power modules, antenna enhancements, and serial logging systems.

In addition, the project benefits from a modular firmware architecture that can be easily updated, modified, and redeployed. The open-source nature of the platform ensures that developers can access libraries, modify core routines, and integrate higher-level data visualization or machine learning modules if required. Thus, OuroMini represents a technically achievable and scalable embedded security monitoring system.

2.4.2 Robustness and Reliability

Reliability is a key factor in the design philosophy of **OuroMini**. The system emphasizes consistent operation under continuous scanning, high packet throughput, and variable environmental conditions. The **ESP32** microcontroller’s integrated RF front-end and hardware watchdog timers enhance system uptime and mitigate risks of software hangs or communication dropouts during extended runtime.

Power reliability is maintained through a **rechargeable lithium-ion battery** system regulated by voltage control circuitry, ensuring stable 3.3V and 5V outputs even under varying load conditions. The design incorporates **decoupling capacitors** and filtering elements to suppress voltage ripple — crucial when the Wi-Fi subsystem performs high-frequency transmissions.

The firmware employs **task prioritization and interrupt-based event handling**, allowing the system to remain responsive during simultaneous serial communication, scanning, and data logging. Furthermore, the simplicity of hardware interconnections (no display, no servo control) enhances overall stability by reducing failure points.

The project has undergone multiple operational tests, confirming that the system maintains reliable serial output and stable performance across repeated scanning cycles, ensuring consistent data acquisition and safe operation.

2.4.3 Economic Feasibility

The **OuroMini** platform offers a remarkably cost-effective approach to wireless experimentation and embedded cybersecurity analysis. Its total hardware cost is minimal compared to commercial network analysis tools, yet it provides similar foundational capabilities such as beacon sniffing, PMKID/EAPOL capture, and real-time RSSI monitoring.

The ESP32-WROOM-32 module — the core processing unit — is widely available at low cost while offering a feature set comparable to more expensive embedded processors. Supporting peripherals, including lithium power modules, perfboards, and connectors, can be locally sourced with minimal expenditure. Since the firmware stack is entirely based on **open-source libraries and toolchains**, there are no licensing or subscription costs associated with its development or deployment.

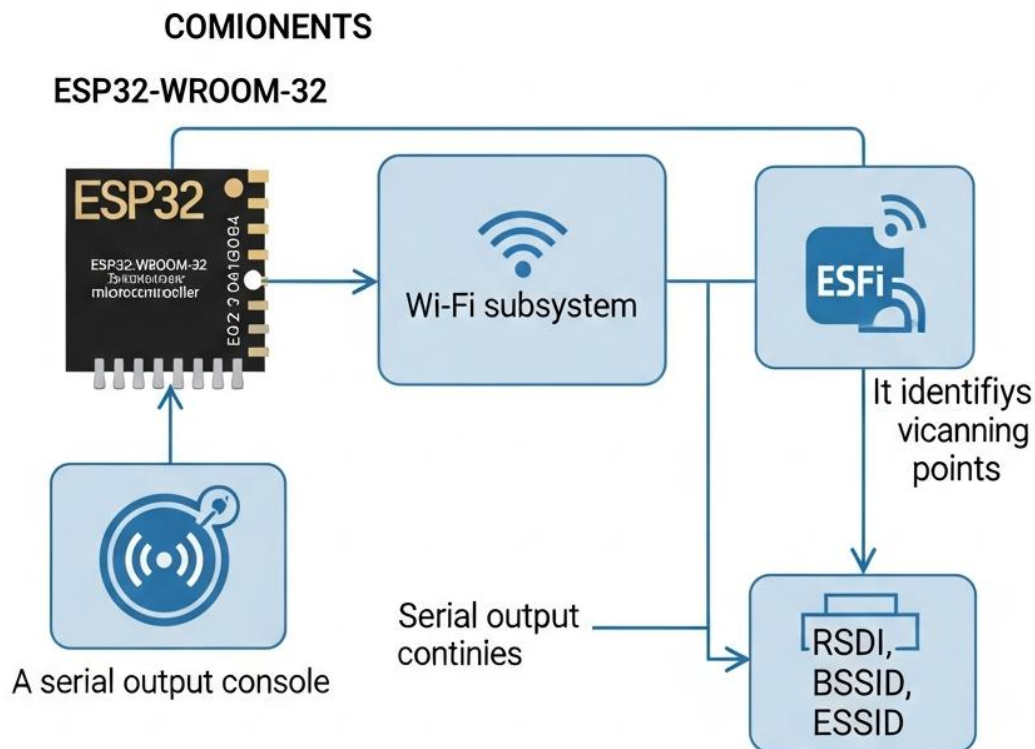
The economic feasibility is further enhanced by its **scalability** — additional modules or sensors (for example, Sub-GHz, BLE, or RF expansions) can be added in future iterations without redesigning the entire hardware. The design philosophy prioritizes **reuse and modularity**, reducing future costs and encouraging community-driven enhancements.

Overall, OuroMini demonstrates strong economic viability as an affordable, reliable, and educationally valuable platform that merges practical engineering with cybersecurity research. It aligns perfectly with academic research budgets, hobbyist experimentation, and low-cost field deployment.

3. Architectural Design

3.1 Modules Design

3.1.1 Wi-Fi Scanning and Packet Capture Module



Description:

This is the core operational module responsible for scanning nearby 2.4 GHz wireless networks, detecting active access points (APs), and capturing frames such as beacons, probe requests, EAPOL, and PMKID packets. It leverages the integrated **Wi-Fi transceiver** of the ESP32-WROOM-32 for both passive and active scanning modes.

Components:

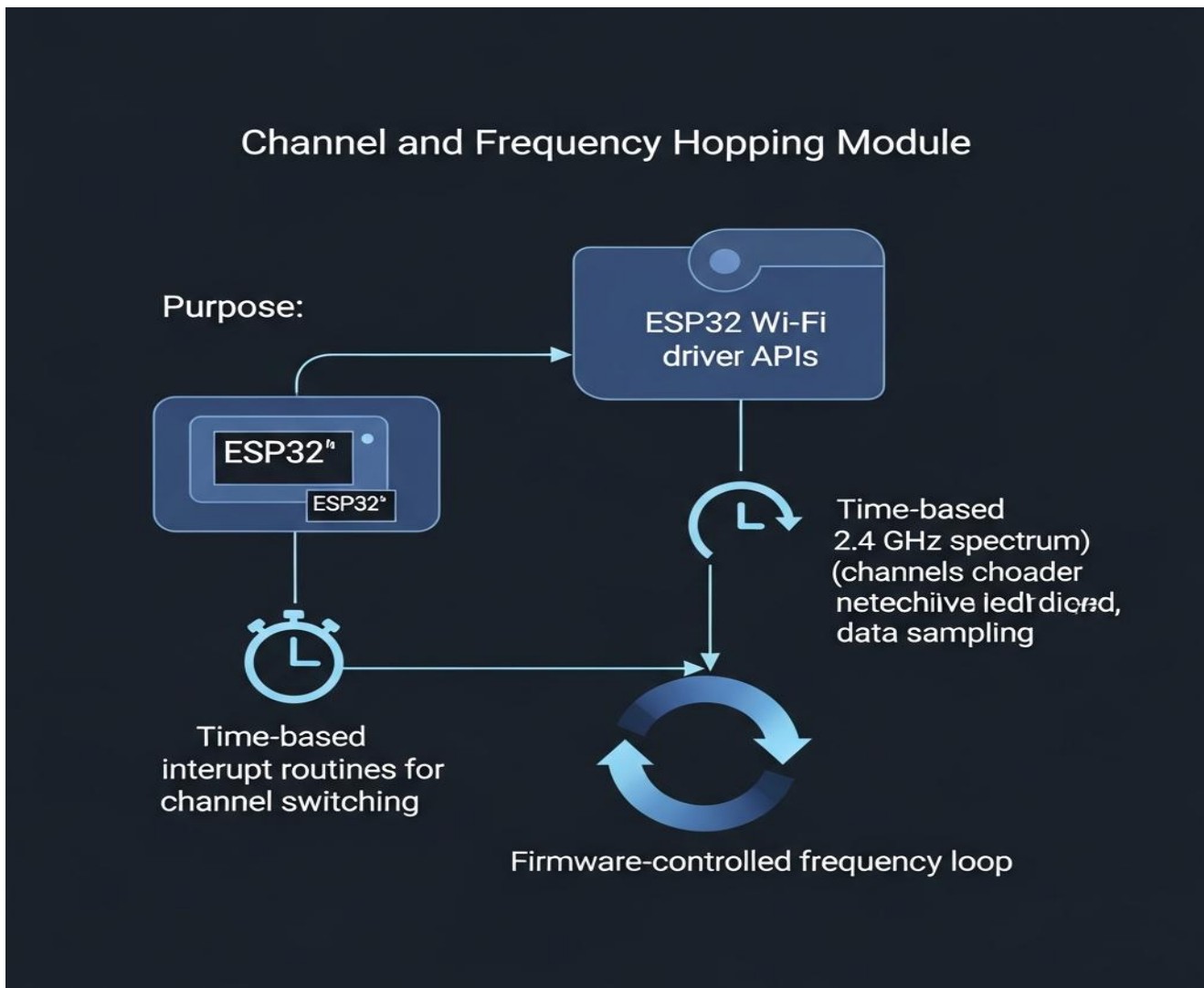
- ESP32-WROOM-32 microcontroller
- Wi-Fi subsystem and antenna interface
- Firmware scanning routines (for beacon, probe, EAPOL, and PMKID packets)
- Serial output console for data visualization

Purpose:

- Identify all visible access points and client devices within range.
- Capture and log wireless packets for later analysis.
- Provide essential data such as **RSSI**, **channel**, **BSSID**, and **ESSID** through serial output.

- Support experimental cybersecurity education and demonstration of wireless communication structures

3.1.2 Channel and Frequency Hopping Module



Description:

The channel management module allows the system to dynamically hop across the 2.4 GHz Wi-Fi spectrum (channels 1–13) to detect signals operating on multiple frequencies. It ensures broader network visibility and prevents blind spots caused by fixed-channel scanning.

Components:

- ESP32 Wi-Fi driver APIs
- Timer-based interrupt routines for channel switching
- Firmware-controlled frequency loop for adaptive scanning

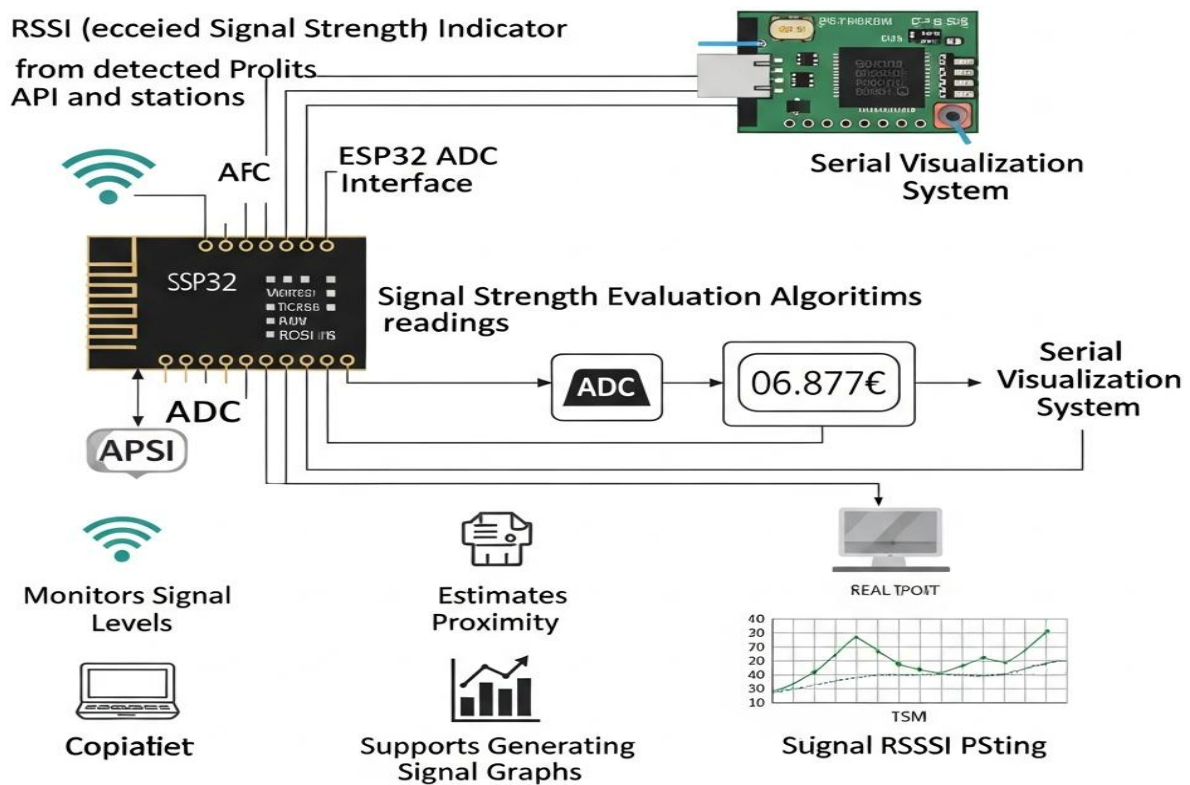
Purpose:

- Cycle through Wi-Fi channels for comprehensive packet detection.
- Ensure even sampling of data across all operational bands.

- Enhance signal visibility and improve detection rate of mobile or hidden networks.
- Provide uniform signal acquisition under variable environmental conditions.

3.1.3 RSSI & Signal Analysis Module

RSSI & Signal Analysis Module



Description:

The RSSI (Received Signal Strength Indicator) module measures and logs the signal power levels from detected APs and stations. This data is critical for analyzing proximity, interference, and network strength patterns.

Components:

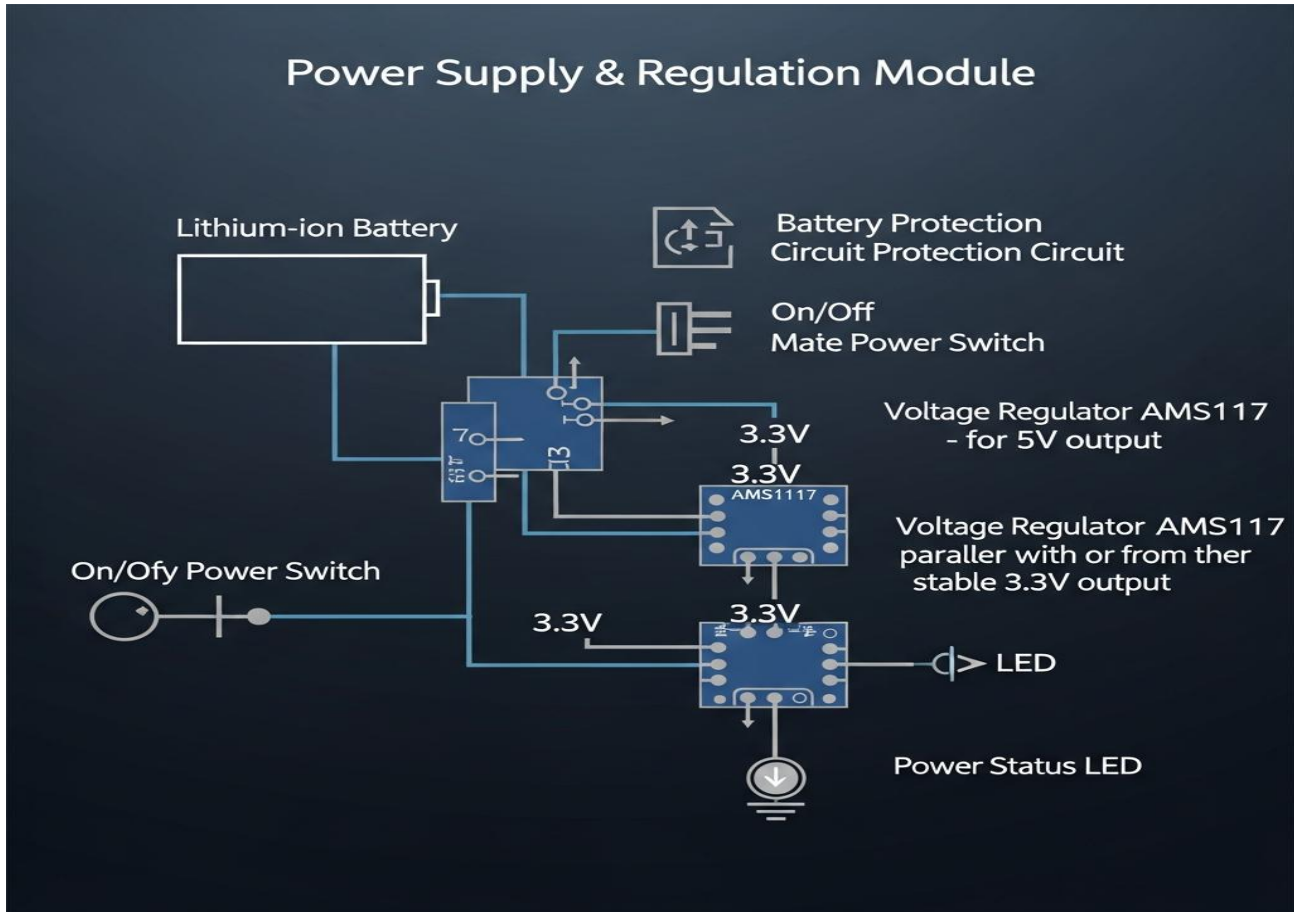
- ESP32 ADC interfaces for internal RSSI computation
- Signal strength evaluation algorithms
- Serial visualization system for real-time RSSI logging

Purpose:

- Continuously monitor wireless signal levels.
- Provide feedback for proximity estimation and interference mapping.
- Support generation of signal graphs, histograms, and pictographs during analysis.

- Aid in research and experimental wireless security testing.

3.1.4 Power Supply & Regulation Module



Description:

This circuit provides stable and efficient power delivery to the ESP32 and all peripheral components. The module converts battery voltage to the appropriate 3.3V and 5V levels required by the core board and external modules.

Components:

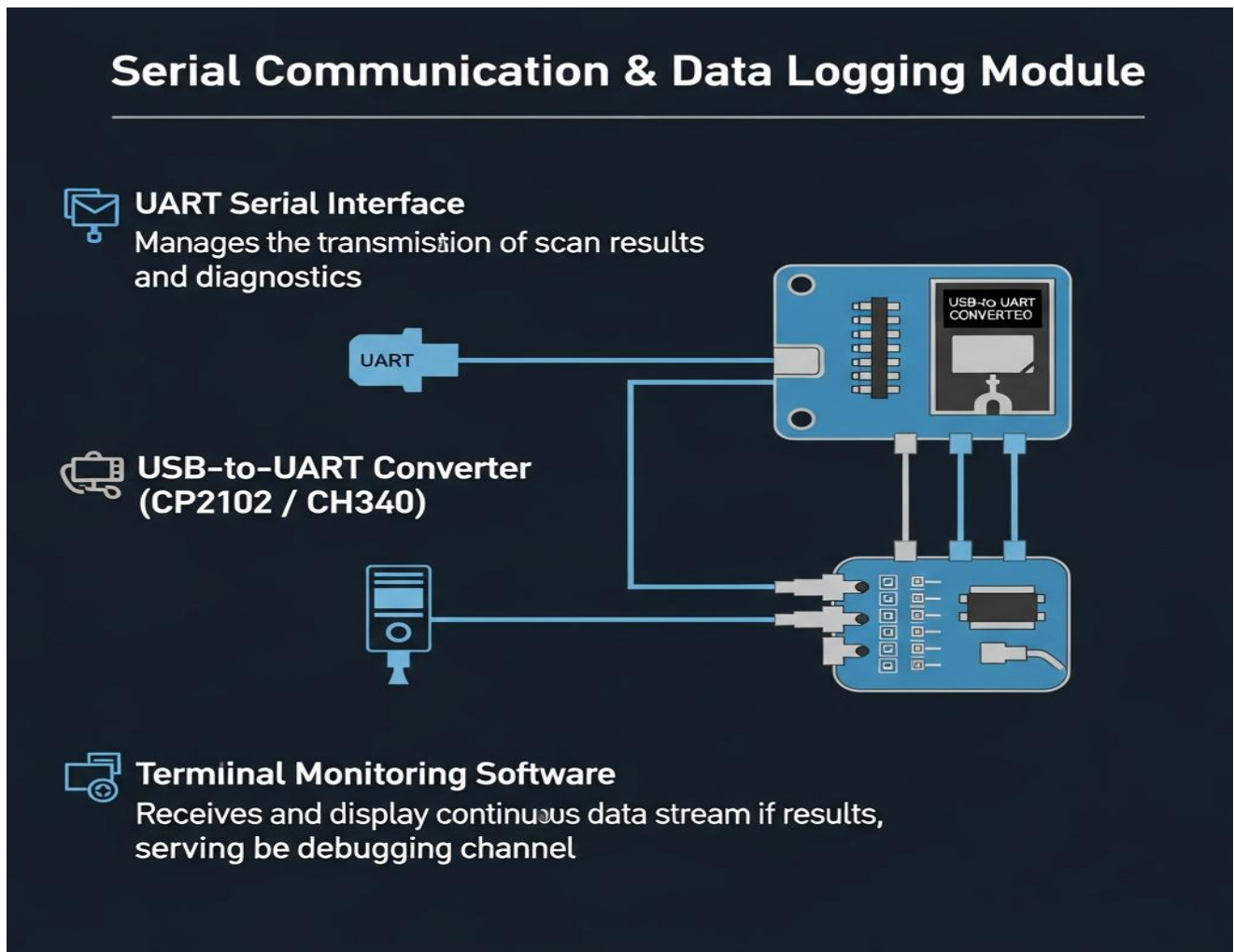
- Lithium-ion battery (3.7V nominal)
- Battery protection circuit and voltage regulators (AMS1117-3.3V / 5V)
- On/Off power switch
- Power status LED indicator

Purpose:

- Supply consistent regulated voltage to prevent brownout resets.
- Protect the circuit from overcharging and discharge-related issues.

- Extend operational uptime while ensuring safety and power stability.

3.1.5 Serial Communication & Data Logging Module



Description:

This module manages the transmission of scan results, captured packets, and diagnostic messages to a host computer or terminal. The ESP32's UART interface is used to relay structured data in real time for analysis or visualization.

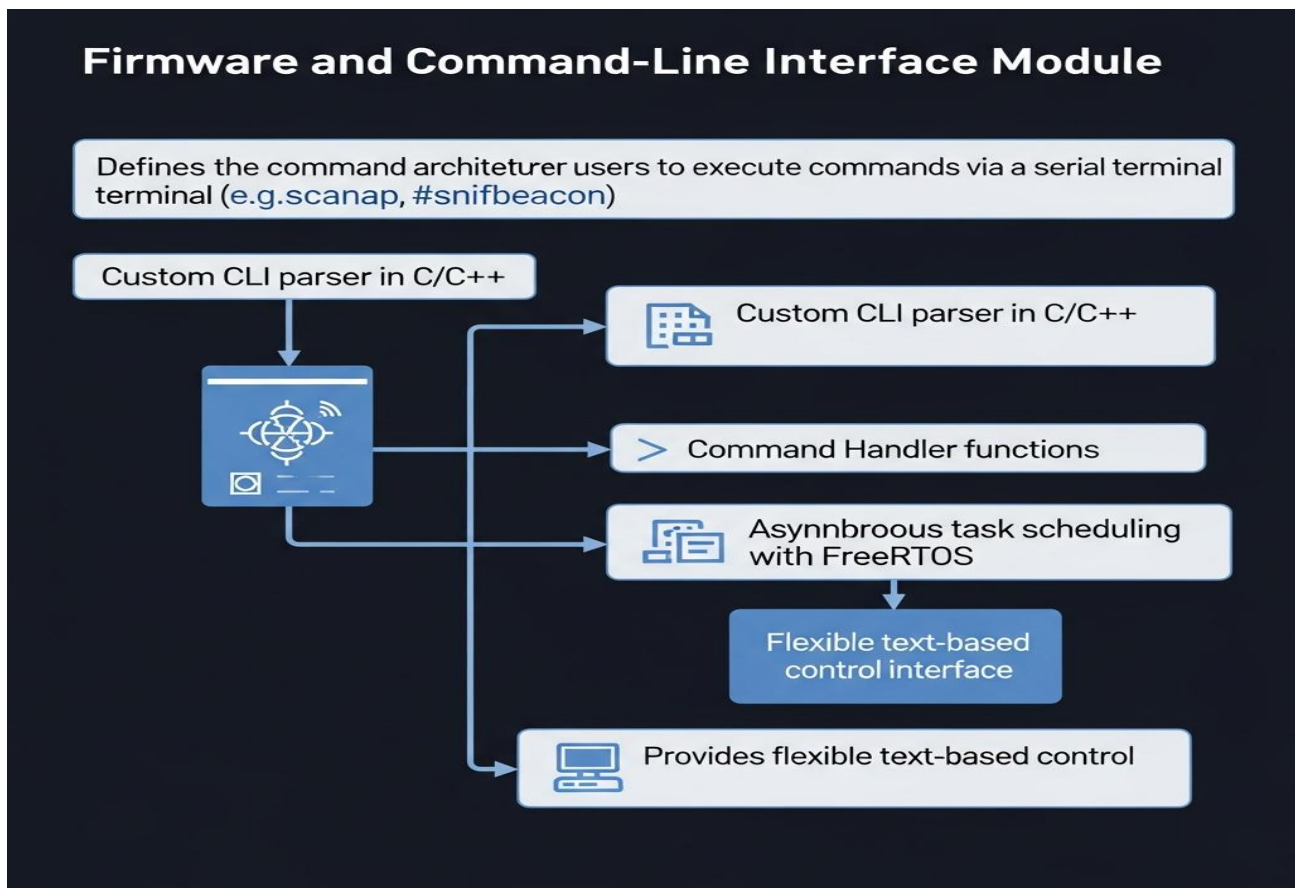
Components:

- UART serial interface
- USB-to-UART converter (CP2102 / CH340)
- Terminal monitoring software (Arduino Serial Monitor, PuTTY, or TeraTerm)

Purpose:

- Provide a continuous data stream of scanning and sniffing results.
- Allow users to visualize results directly or record them for offline analysis.
- Serve as a debugging and monitoring channel for firmware diagnostics.

3.1.6 Firmware and Command-Line Interface Module



Description:

This software module defines the command architecture and interactive control system that allows users to execute scanning, sniffing, and logging commands via the serial terminal. It interprets user inputs such as #scanap, #sniffbeacon, or #sniffpmkid and triggers corresponding firmware routines.

Components:

- Custom CLI parser in C/C++ for ESP32 firmware
- Command handler functions for Wi-Fi operations
- Asynchronous task scheduling using FreeRTOS

Purpose:

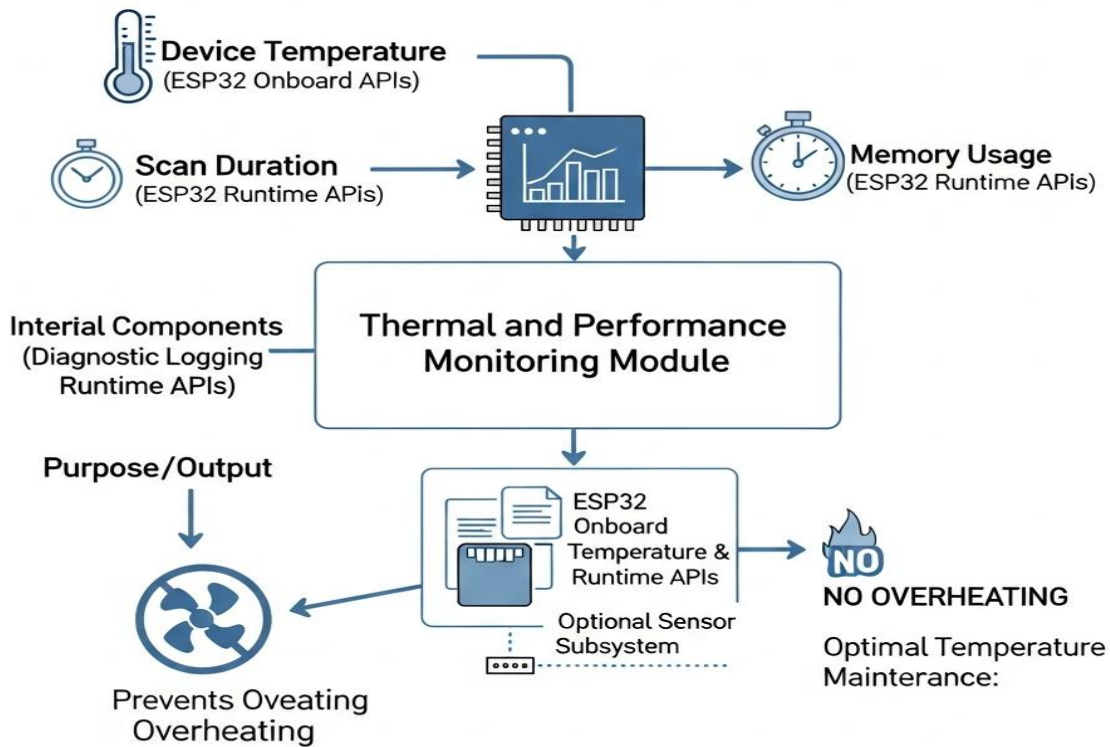
- Provide a flexible and text-based control interface.

- Allow users to initiate and stop processes dynamically (`stopscan`, `list -a`, `list -c`, etc.).
- Enable real-time configuration, debugging, and control without external GUIs.
- Ensure firmware-level modularity for future toolchain integration (web dashboard or Bluetooth console).

3.1.7 Thermal and Performance Monitoring Module

Central Module

Input /Monitoring



Description:

The performance management module monitors device temperature, scan duration, and memory usage to prevent overheating and performance degradation during prolonged use.

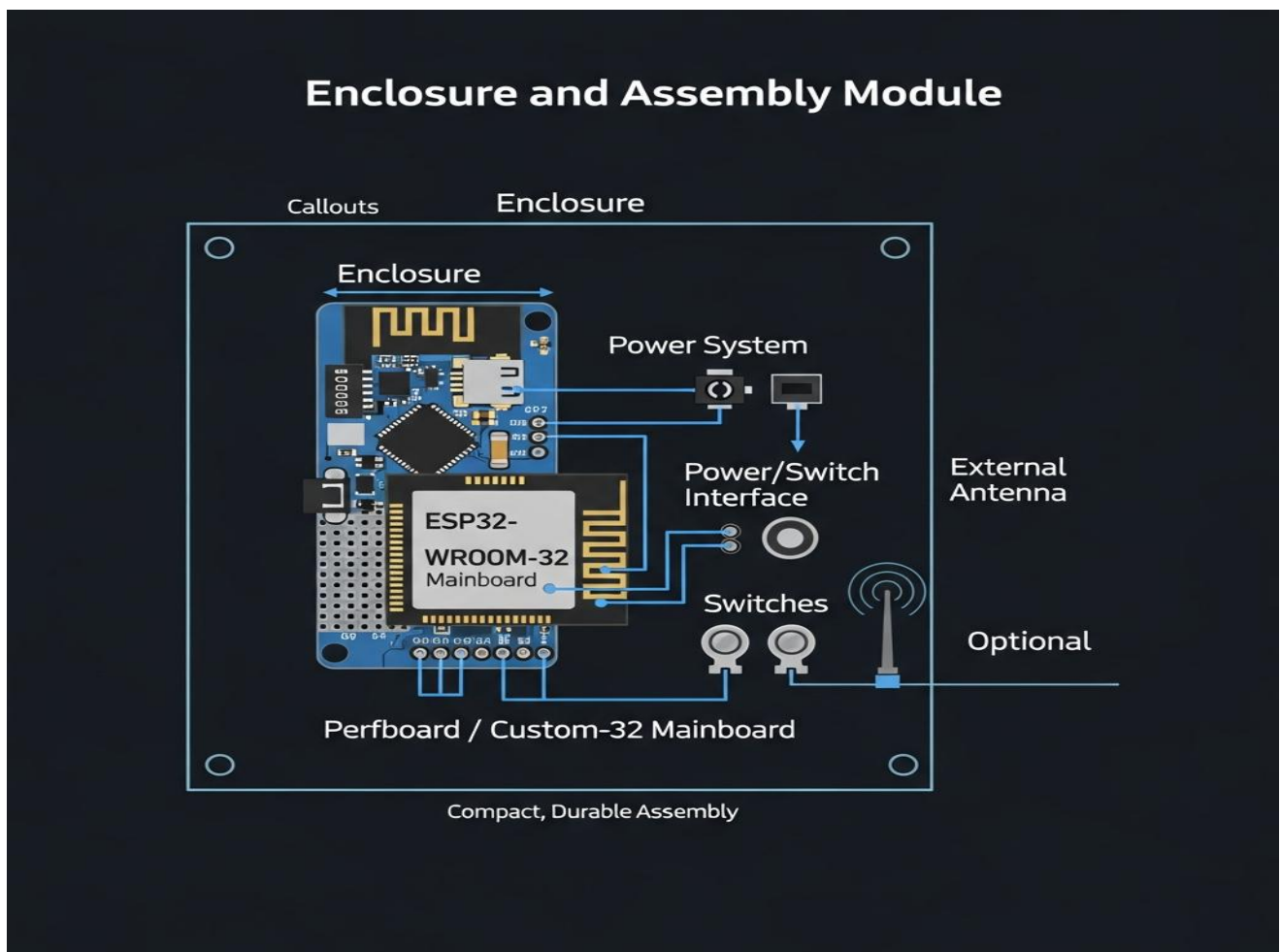
Components:

- ESP32 onboard temperature and runtime monitoring APIs
- Diagnostic logging subsystem
- Optional sensor header for thermistor or DS18B20

Purpose:

- Maintain optimal temperature during continuous scanning.
- Prevent memory overflow or Wi-Fi driver saturation.
- Provide feedback loops for firmware-based performance throttling or sleep modes.

3.1.8 Enclosure and Assembly Module



Description:

The physical framework of **OuroMini** is built on a lightweight **perfboard or custom PCB**, optimized for compactness and modular wiring. It integrates the ESP32 board, Li-ion power system, switches, and antennas in a minimalistic enclosure.

Components:

- ESP32-WROOM-32 mainboard
- Perfboard-based wiring harness
- Compact power and switch interface
- External antenna (optional for range extension)

Purpose:

- Ensure compact, durable, and portable assembly.
 - Facilitate heat dissipation and accessibility during demonstrations.
 - Provide a professional finish for academic, research, and field testing presentations.
-

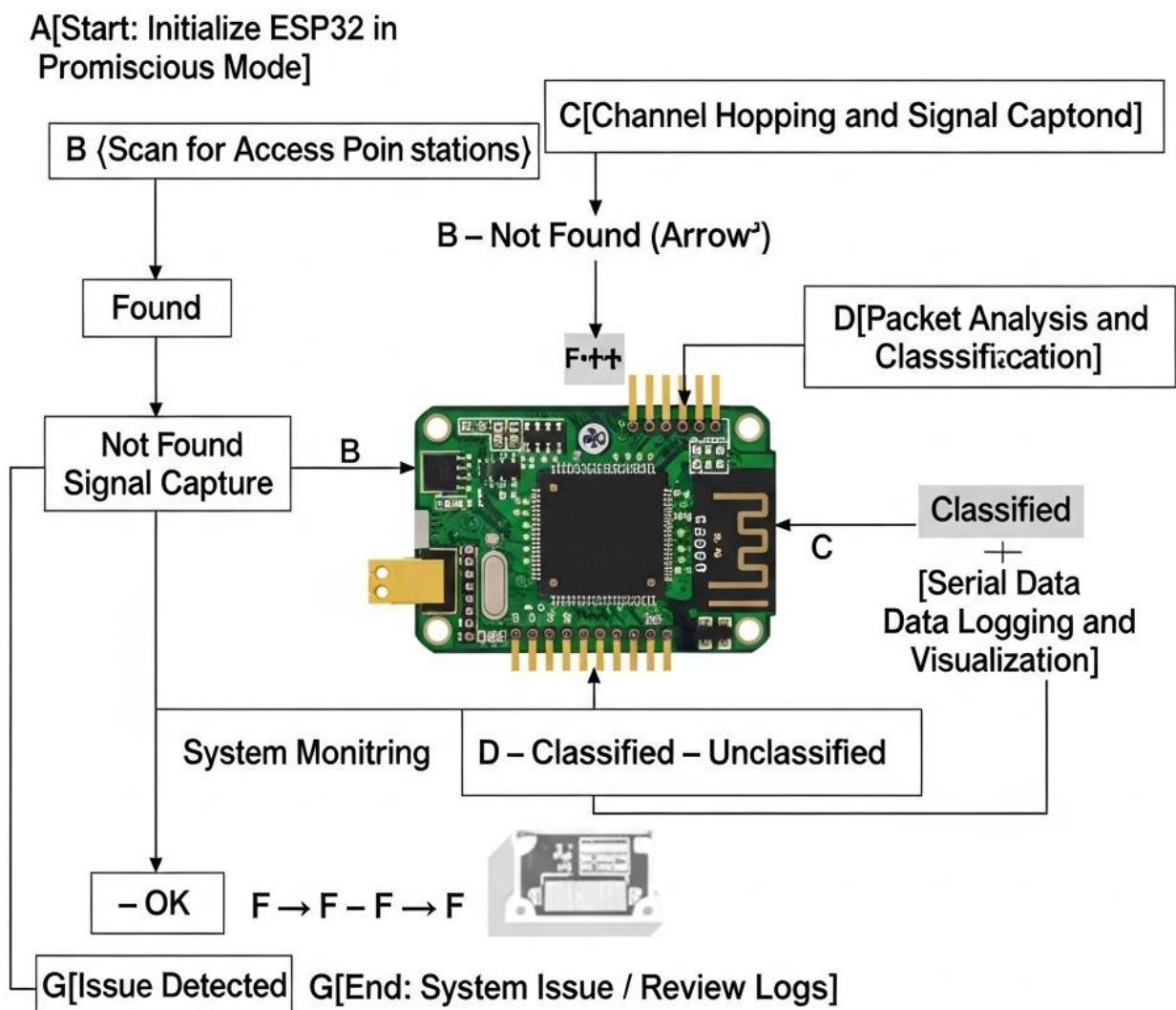
3.2 Method & Algorithm Design

3.2.1 Method

The **OuroMini** system employs an **Embedded Passive Wireless Analysis Method**, designed to enable autonomous and real-time Wi-Fi reconnaissance without the need for external computation. This approach merges firmware-level packet capture with intelligent serial processing and structured logging. The methodology is centered around **efficiency, modularity, and precision**, ensuring continuous and non-intrusive wireless monitoring.



Embdded Passive Wireless Analysis Method (OuroMini)



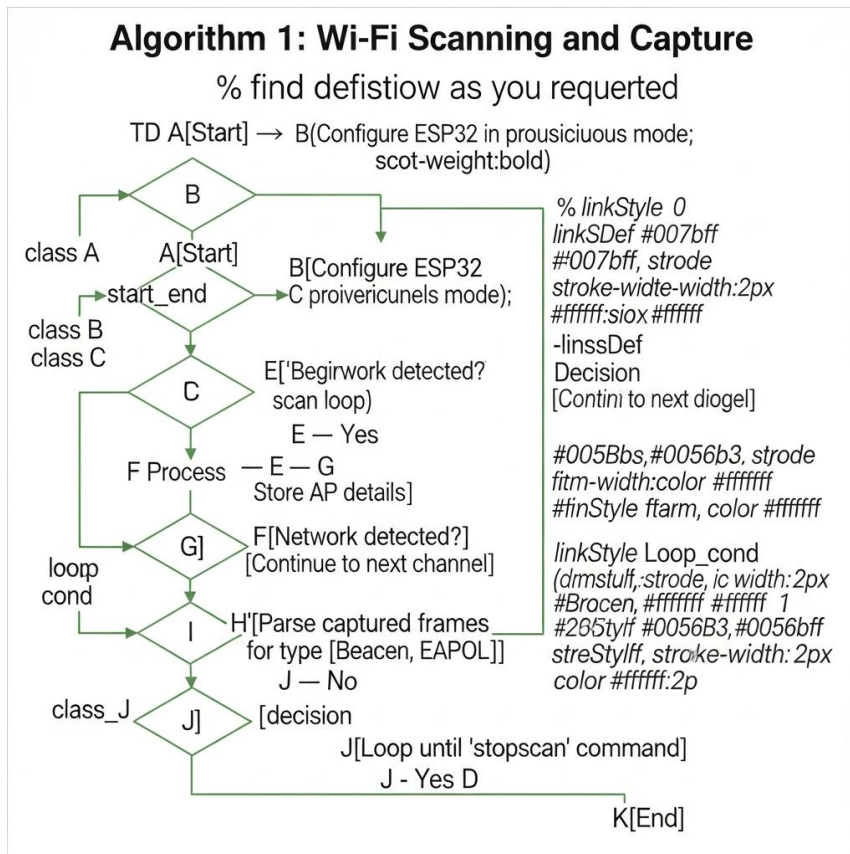
OuroMini's operation follows this conceptual flow:

1. **Initialize ESP32 in Promiscuous Mode:**
The firmware configures the ESP32-WROOM-32 to capture all 802.11 frames within range.
2. **Scan for Access Points (APs) and Stations:**
Using firmware commands (#scanap, #sniffbeacon, #sniffprobe), the system detects active networks and connected clients.
3. **Channel Hopping and Signal Capture:**
The firmware cycles through available Wi-Fi channels (1–13) to ensure full spectrum coverage.
4. **Packet Analysis and Classification:**
Captured frames are parsed to identify beacon frames, probe requests, and EAPOL/PMKID handshakes.
5. **Serial Data Logging and Visualization:**
Processed results, including RSSI, BSSID, ESSID, and frame type, are output through the serial terminal for visualization or export.
6. **System Monitoring:**
Throughout operation, the firmware monitors temperature, signal load, and battery voltage for reliability and sustained uptime.

This modular method ensures **continuous data flow**, **low-latency response**, and **autonomous field performance**, making OuroMini a self-contained, research-grade network visualization platform.

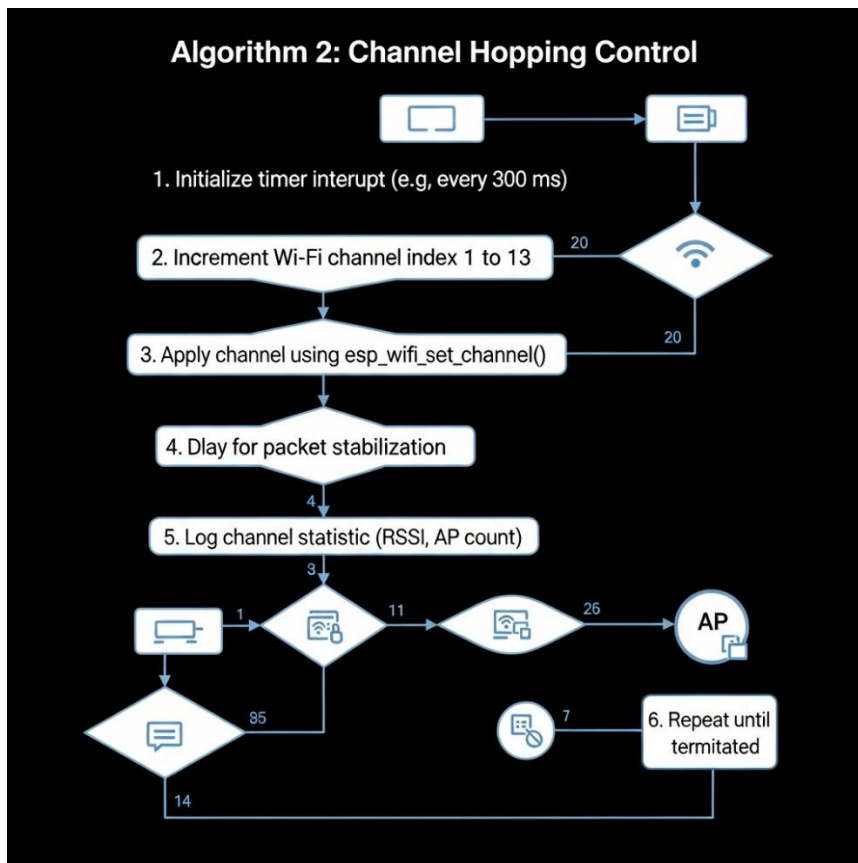
3.2.2 Algorithms

Algorithm 1: Wi-Fi Scanning and Capture Algorithm



1. **Start System Initialization**
2. Configure ESP32 in promiscuous (RX) mode
3. Set scanning interval and channel hopping timer
4. Begin channel scan loop
 - IF network detected → Store AP details (SSID, BSSID, RSSI, Channel)
 - ELSE → Continue scanning next channel
5. Parse captured frames to identify frame type
 - IF frame = Beacon → Record AP information
 - IF frame = EAPOL → Store handshake metadata
6. Output structured data to Serial Monitor
7. Loop until “stopscan” command received
8. **End.**

Algorithm 2: Channel Hopping Control

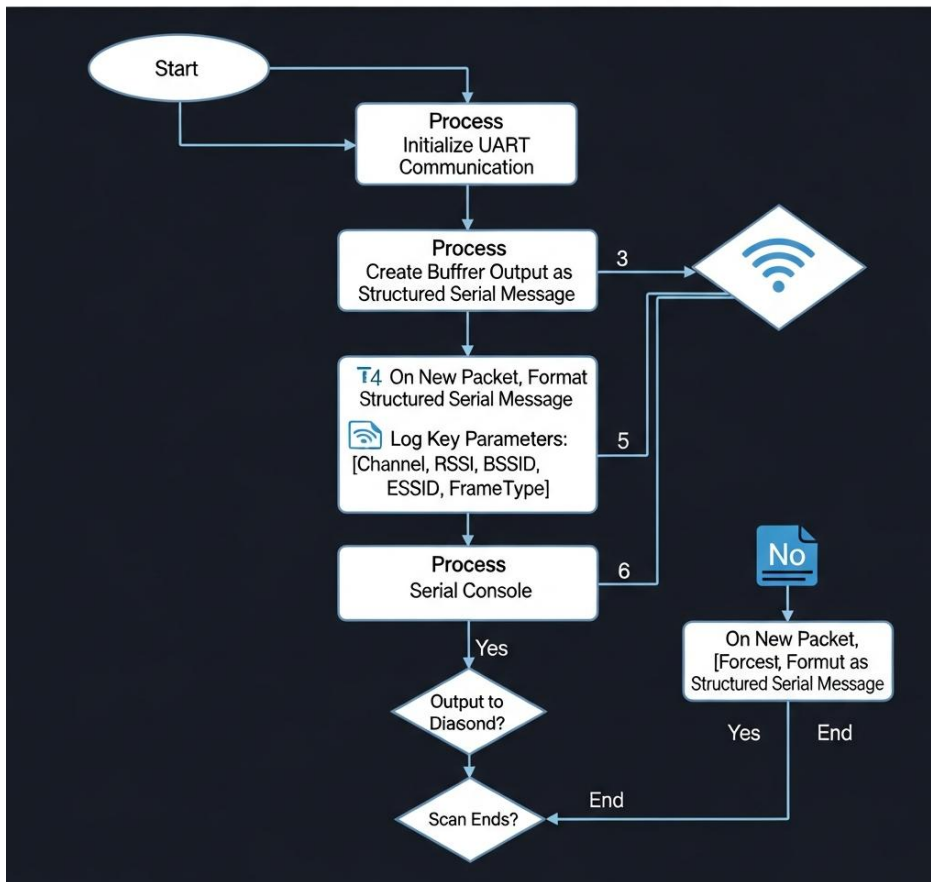


1. Initialize timer interrupt (e.g., every 300 ms)
2. Increment Wi-Fi channel index (1 → 13)
3. Apply channel using `esp_wifi_set_channel()`
4. Delay for packet stabilization
5. Log channel statistics (RSSI, AP count, activity level)
6. Repeat until terminated by command input

This ensures real-time coverage of all frequencies with minimal packet overlap and consistent performance during long-duration scanning sessions.

Algorithm 3: Data Logging and Display Algorithm

Algorithm 3: Data Logging and Display



1. Initialize UART communication
2. Create buffer for incoming frame data
3. On new packet → Format output as structured serial message
4. Log key parameters: {Channel, RSSI, BSSID, ESSID, FrameType}
5. Output to serial console for visualization
6. Repeat until scan ends or system resets

This algorithm manages the low-level data I/O pipeline and serial communication used for producing visual datasets and pictographs from the results.

3.3 Project Architecture

3.3.1 Architecture

The **OuroMini** architecture is a **three-layer embedded monitoring framework**, optimized for minimal hardware, high performance, and modular code design.

Layers:

- **Input Layer** –
Captures environmental and wireless data via the ESP32's internal Wi-Fi transceiver in promiscuous mode.
Inputs include frame payloads, signal strength, and channel identifiers.
- **Processing Layer** –
The ESP32 firmware executes multi-threaded FreeRTOS tasks to perform packet analysis, frame classification, and data structuring.
It handles filtering, noise reduction, and real-time signal correlation across channels.
- **Output Layer** –
Transmits parsed data to the serial interface or external logging dashboard for visualization and interpretation.
The layer also manages LED indicators and future extensions for OLED or Bluetooth console output.

Data Flow Summary:

1. **ESP32** initializes and begins channel-based scanning.
2. Captured packets are pushed to the processing queue.
3. Firmware classifies frame types and extracts relevant metadata.
4. Data is structured and transmitted to the output terminal.
5. Optional modules visualize RSSI, client relations, or network density in graphs.

3.3.1 Architecture Diagram (Description)

OuroMini Architecture Diagram

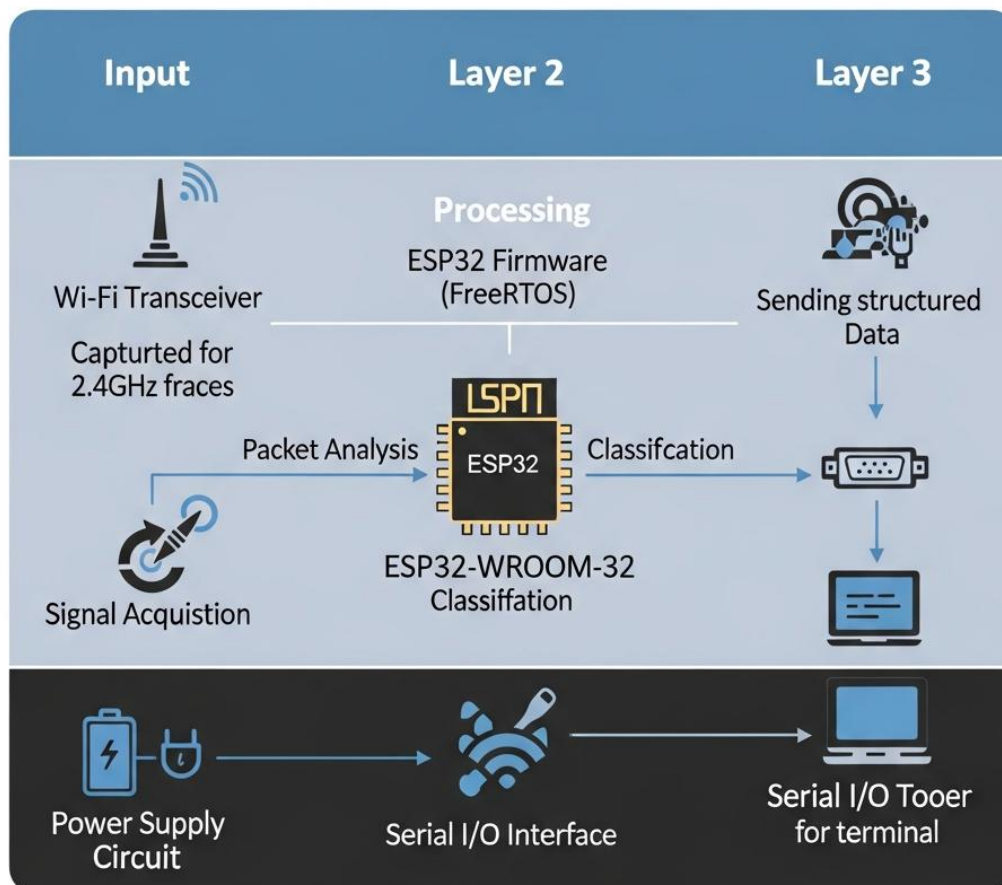


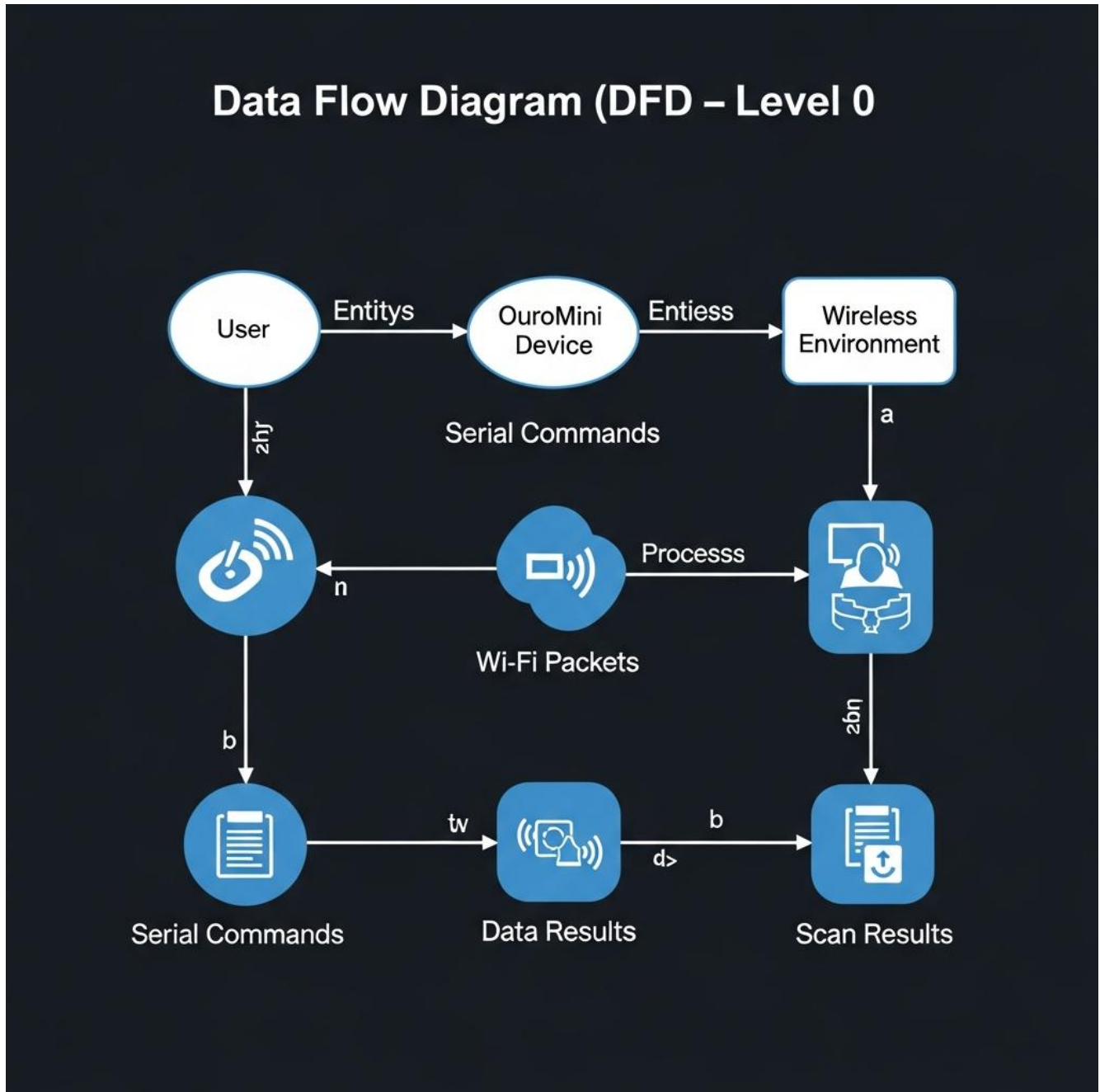
Diagram Components:

- **ESP32-WROOM-32** (Central Processing Unit)
- **Wi-Fi Transceiver Subsystem** → Captures 2.4GHz frames
- **Firmware Processing Layer** → Parses and classifies packets
- **Serial I/O Interface** → Sends structured data to terminal
- **Power Supply Circuit** → Li-ion input, regulators, and switches

The diagram will visually depict data flow from Wi-Fi signal acquisition to serial output generation, showing all internal processing blocks.

3.3.2 Data Flow Diagram (DFD)

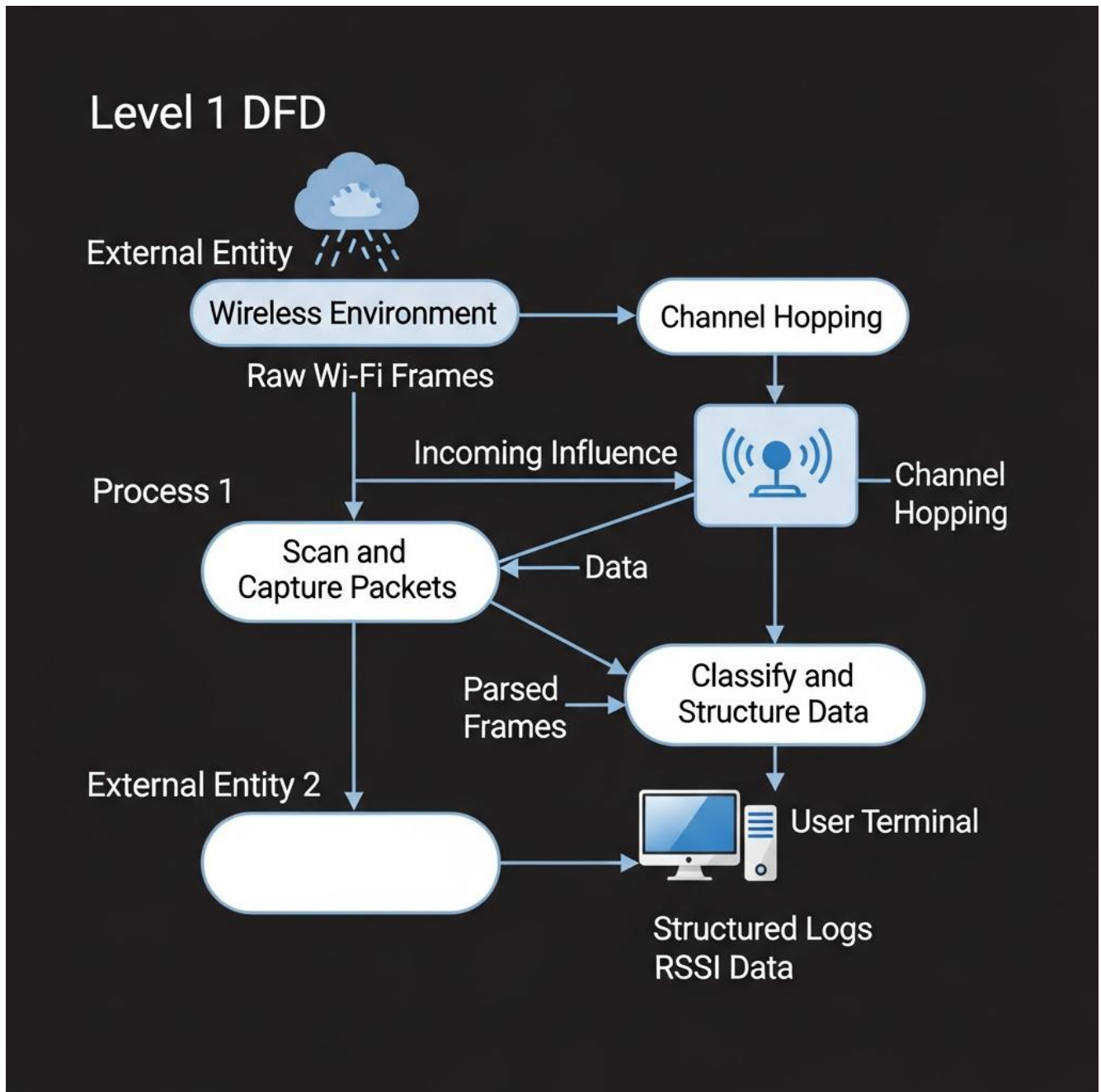
Level 0 (Context Diagram)



Shows interaction between the **User**, **OuroMini Device**, and **Wireless Environment**:

- User sends serial commands (e.g., #scanap, #sniffpmkid)
- Device scans and captures packets
- Results are output to terminal/logging software

Level 1 (Detailed DFD)



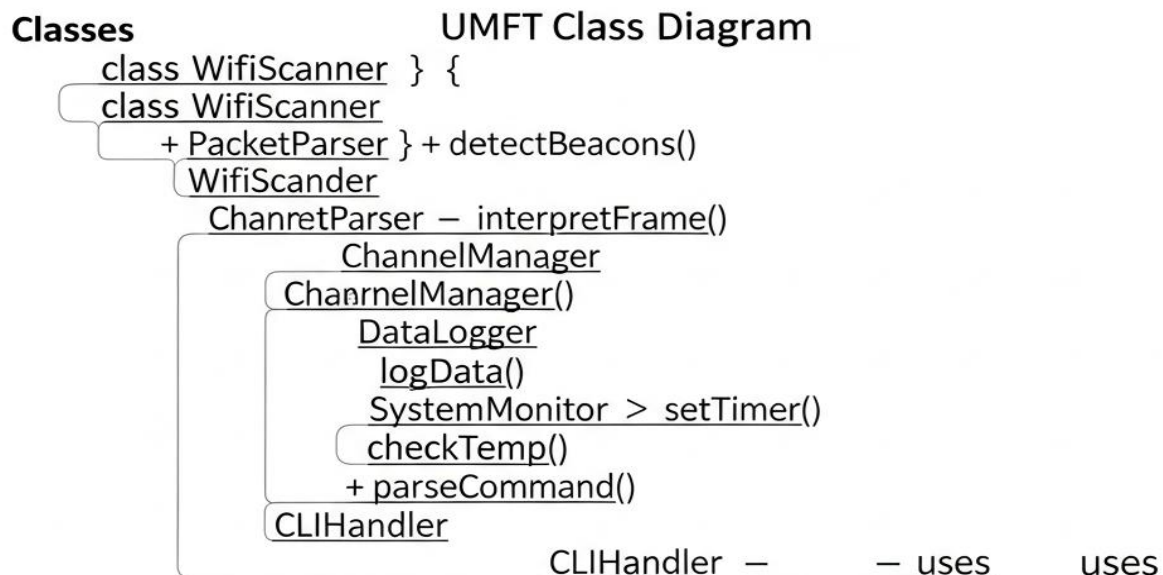
- **Input:** Raw Wi-Fi frames
- **Processing:** Packet parsing, channel mapping, classification
- **Output:** Structured logs, RSSI data, beacon lists

3.3.3 Class Diagram

Primary Classes:

- `WiFiScanner` — handles scanning and beacon detection
- `PacketParser` — interprets raw frame data
- `ChannelManager` — cycles channels and maintains timing
- `DataLogger` — manages structured output
- `SystemMonitor` — tracks performance and voltage
- `CLIHandler` — interprets user commands

These classes interact under a lightweight object-oriented firmware model (in C++/Arduino style), ensuring modular code and easy debugging.



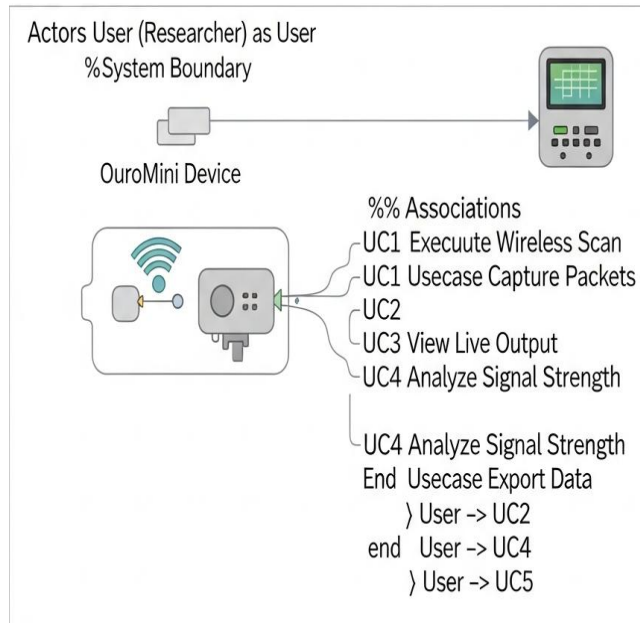
Optional: Add more classes or details
wr helper classee:

For example : there more classes or details of details if needed
Class for represeenting packet for Pracket helper classes

```

Class for represemng data }>
Class WifiSander — WifiPacket {
  { + WifiPacket {
    + payload — rssi {
      checktVoltage() }
  }
  Class for BeaconData : produces >
    — macAddress — Ssid — rsid — 113417° timestamp }
    { macAddress — rssi — rsii +56403 — 47777° timestamp }
  WifiScanner — BeaconData >
  
```

3.3.4 Use Case Diagram



Actors:

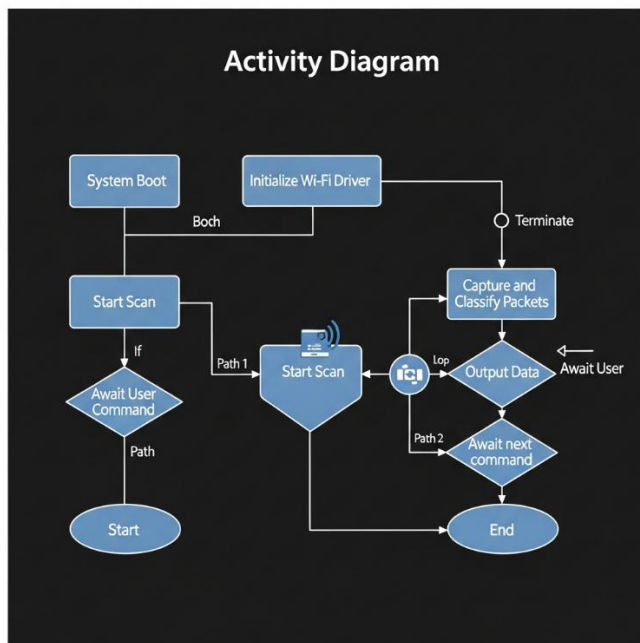
- **User (Researcher or Analyst)**
- **OuroMini Device**

Use Cases:

- Execute wireless scan
- Capture packets in real time
- View live output via serial
- Analyze signal strength and frame type
- Export data for visualization

3.3.5 Activity Diagram

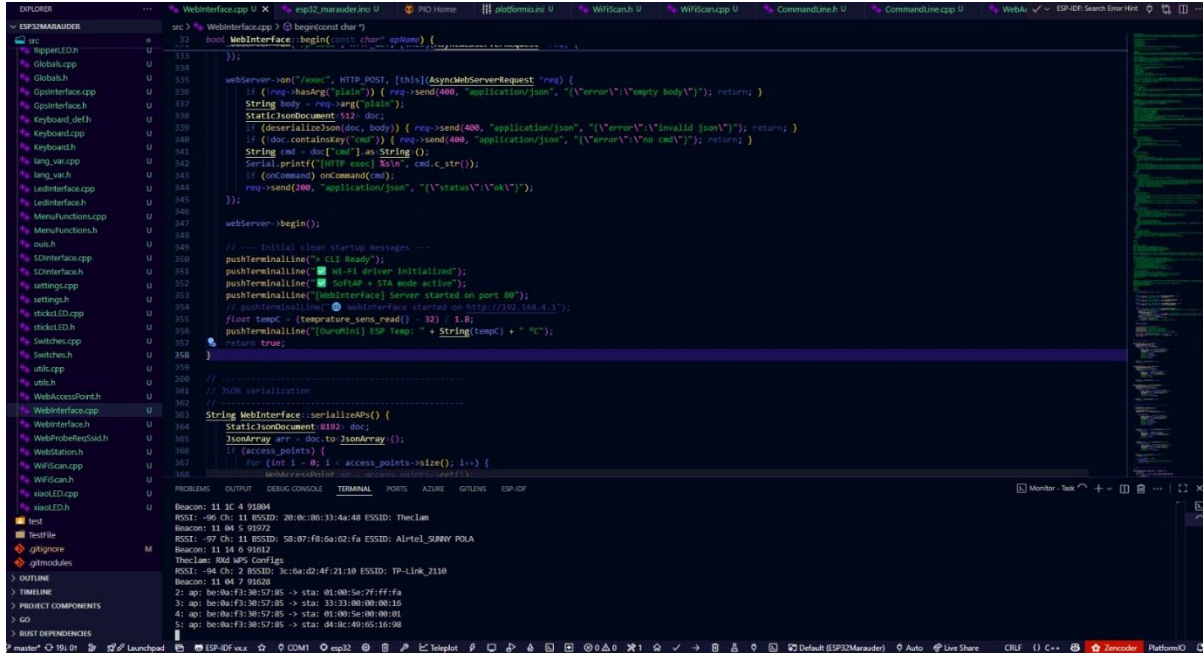
Describes the operational workflow of OuroMini:



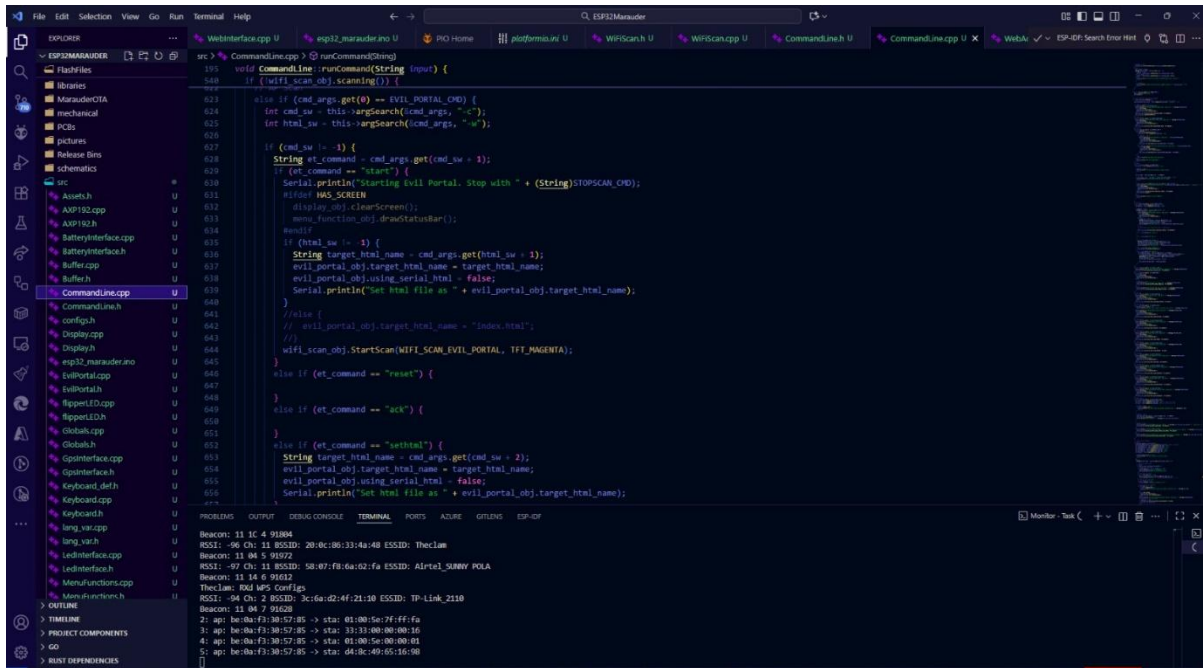
1. System boot
2. Wi-Fi driver initialization
3. User command input
4. Start scan
5. Capture and classify packets
6. Output data
7. Await next command or terminate

4. Implementation

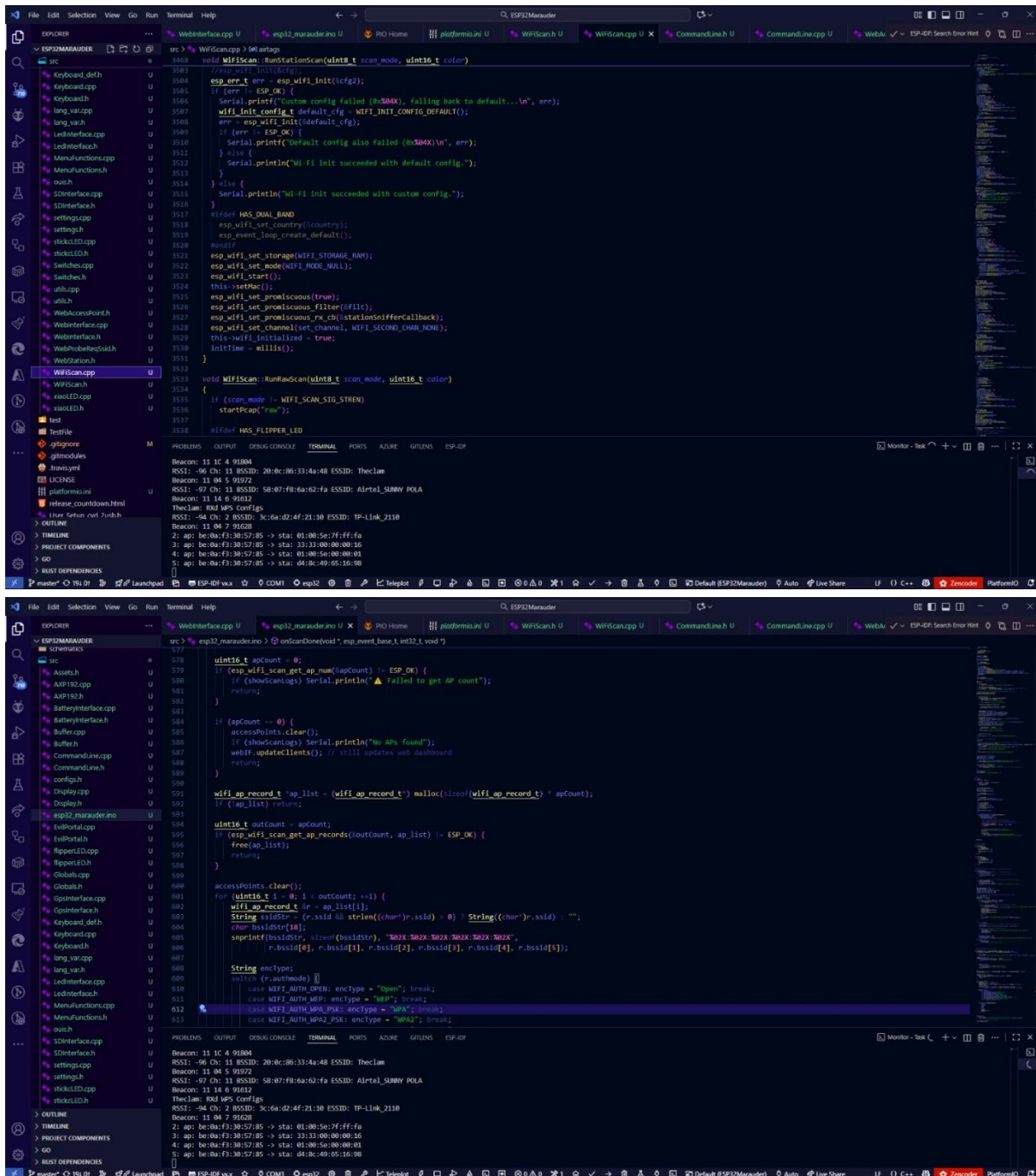
4.1 Coding Blocks



```
src > WebInterface.cpp > begin(const char*)
132 bool WebInterface::begin(const char* name) {
133     // ...
134     // ...
135     webServer->on("/exec", HTTP_POST, [this](AsyncWebServerRequest *req) {
136         if (req->hasArg("plain")) { req->send(400, "application/json", "{\"error\":\"empty body\"}"); return; }
137         String body = req->arg("plain");
138         StaticJsonDocument<512> doc;
139         if (deserializeJson(doc, body)) { req->send(400, "application/json", "{\"error\":\"invalid json\"}"); return; }
140         if (doc.containsKey("cmd")) { req->send(400, "application/json", "{\"error\":\"no cmd\"}"); return; }
141         String cmd = doc["cmd"].as<String>();
142         Serial.printf("[HTTP exec] %s\n", cmd.c_str());
143         if (onCommand(cmd)) {
144             req->send(200, "application/json", "{\"status\":\"ok\"}");
145         }
146     });
147     webServer->begin();
148     // ...
149     // --- Initial clean startup messages ---
150     pushTermLineLine("CLI Ready");
151     pushTermLineLine("Wi-Fi driver initialized");
152     pushTermLineLine("SoftAP + STA mode active");
153     pushTermLineLine("[WebInterface] Server started on port 80");
154     // pushTermLineLine("WebInterface started on http://192.168.4.1");
155     float tempc = (temperature_sens.read() > 32) ? 1.8 : 0;
156     pushTermLineLine("[DuroMini] ESP Temp: " + String(tempc) + " °C");
157     return true;
158 }
159 // ...
160 // JSON serialization
161 String WebInterface::serializeJson() {
162     StaticJsonDocument<1024> doc;
163     JsonArray arr = doc.to<JsonArray>();
164     if (access_points) {
165         for (int i = 0; i < access_points->size(); i++) {
166             WebInterface::serializeJson(doc, i);
167         }
168     }
169     return arr.toString();
170 }
171 // ...
172 JsonObject WebInterface::deserializeJson(const char* json) {
173     StaticJsonDocument<1024> doc;
174     if (deserializeJson(doc, json)) {
175         return doc.as<JsonObject>();
176     }
177     return JsonObject();
178 }
179 // ...
180 Beacon: 11 1C 4 91804
181 RSSI: -96 Ch: 11 BSSID: 20:0E:06:33:4A:48 ESSID: TheClan
182 Beacon: 11 04 5 91972
183 RSSI: -97 Ch: 11 BSSID: 58:07:F8:6A:D2:FA ESSID: Airtel_SUNNY POLA
184 Beacon: 11 14 6 91612
185 TheClan: R0d WPS Config
186 RSSI: -94 Ch: 2 BSSID: 3C:6A:D2:4F:21:10 ESSID: TP-Link_2110
187 Beacon: 11 04 7 91628
188 2: ap: be:ba:f3:30:57:85 -> sta: 01:00:5e:7f:ff:fa
189 3: ap: be:ba:f3:30:57:85 -> sta: 33:33:00:00:00:10
190 4: ap: be:ba:f3:30:57:85 -> sta: 01:00:5e:00:00:01
191 5: ap: be:ba:f3:30:57:85 -> sta: d4:8c:40:05:16:90
```



```
src > CommandLine.cpp > runCommand(String input)
195 void CommandLine::runCommand(String input) {
196     if (input.startsWith("start")) {
197         if (cmd_args.get(0) == EVIL_PORTAL_CMD) {
198             for (int i = 1; i < cmd_args.size(); i++) {
199                 if (cmd_args[i].startsWith("html")) {
200                     String et_command = cmd_args.get(i+1);
201                     if (et_command == "start") {
202                         Serial.println("Starting Evil Portal. Stop with " + (String)STOPSCAN_CMD);
203                         if (wifiscan_obj.startScan()) {
204                             menu_function_obj.drawStatusbar();
205                             if (i < (cmd_sw - 1)) {
206                                 String target_html_name = cmd_args.get(i+2);
207                                 evil_portal_obj.target_html_name = target_html_name;
208                                 evil_portal_obj.using_serial_html = false;
209                                 Serial.println("Set html file as " + evil_portal_obj.target_html_name);
210                             }
211                         }
212                     }
213                 }
214             }
215         }
216         if (cmd_sw == 1) {
217             if (et_command == "stop") {
218                 Serial.println("Stopping Evil Portal");
219                 if (wifiscan_obj.stopScan()) {
220                     menu_function_obj.drawStatusbar();
221                 }
222             }
223             if (et_command == "scan") {
224                 String target_html_name = cmd_args.get(i+2);
225                 evil_portal_obj.target_html_name = target_html_name;
226                 evil_portal_obj.using_serial_html = false;
227                 Serial.println("Set html file as " + evil_portal_obj.target_html_name);
228             }
229         }
230     }
231     if (input.startsWith("sethtml")) {
232         String target_html_name = cmd_args.get(i+2);
233         evil_portal_obj.target_html_name = target_html_name;
234         evil_portal_obj.using_serial_html = false;
235         Serial.println("Set html file as " + evil_portal_obj.target_html_name);
236     }
237     if (input.startsWith("ack")) {
238         Serial.println("Ack");
239     }
240 }
241 // ...
242 Beacon: 11 1C 4 91804
243 RSSI: -96 Ch: 11 BSSID: 20:0E:06:33:4A:48 ESSID: TheClan
244 Beacon: 11 04 5 91972
245 RSSI: -97 Ch: 11 BSSID: 58:07:F8:6A:D2:FA ESSID: Airtel_SUNNY POLA
246 Beacon: 11 14 6 91612
247 TheClan: R0d WPS Config
248 RSSI: -94 Ch: 2 BSSID: 3C:6A:D2:4F:21:10 ESSID: TP-Link_2110
249 Beacon: 11 04 7 91628
250 2: ap: be:ba:f3:30:57:85 -> sta: 01:00:5e:7f:ff:fa
251 3: ap: be:ba:f3:30:57:85 -> sta: 33:33:00:00:00:10
252 4: ap: be:ba:f3:30:57:85 -> sta: 01:00:5e:00:00:01
253 5: ap: be:ba:f3:30:57:85 -> sta: d4:8c:40:05:16:90
```

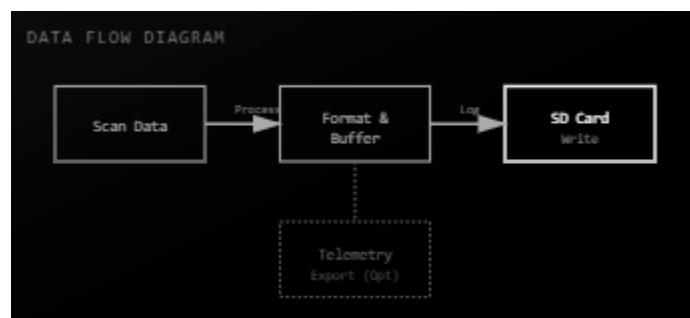
Screenshots of Core code

4.2 Sample Code

SD Logging and Data Management

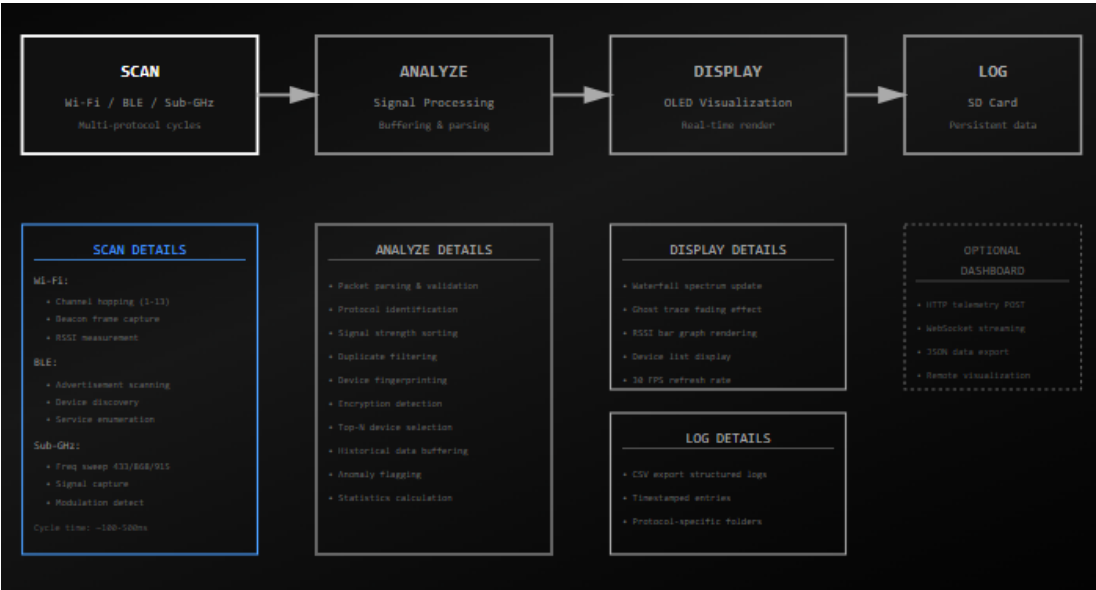


The SD card structure shows organized data logs for different modules — WiFi, BLE, SubGHz, IR, NFC, and telemetry — each storing scans, captures, and analysis files. It also includes configuration (`config.ini`) and documentation (`README.txt`) files for system setup and usage.



The diagram shows that scan data is processed and formatted before being buffered, then logged to the SD card for storage. Optionally, telemetry data can be exported from the buffer for external monitoring or analysis.

Scanning and Monitoring of flow



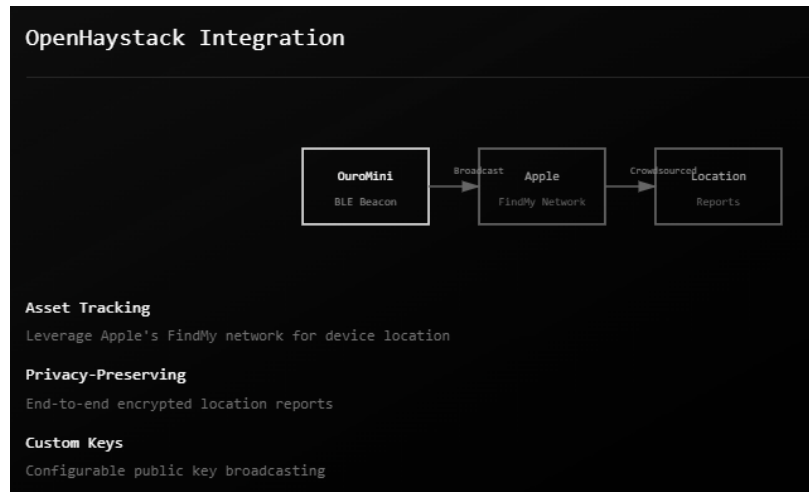
The diagram illustrates a four-stage process: **Scan**, **Analyze**, **Display**, and **Log**. It captures Wi-Fi, BLE, and Sub-GHz signals, processes them for protocol detection and signal analysis, and displays results in real-time on an OLED screen. Data is logged to the SD card with structured, timestamped records. An optional dashboard supports telemetry export, remote monitoring, and JSON data streaming.

USB and connectivity features



1. USB / BADUSB

- Uses ESP32 USB for HID (keyboard) emulation and storage functions.
- Supports **DuckyScript** for automated keyboard payloads in security demos.
- Can act as a **USB mass storage device** for data exfiltration demonstrations.
- Offers **Serial (CDC/ACM)** communication for debugging and telemetry.



2. OpenHaystack Integration

- Integrates with Apple's **Find My network** using BLE beacon broadcasting.
- Enables **asset tracking** through crowdsourced Apple device networks.
- Provides **privacy-preserving**, end-to-end encrypted location reporting.
- Allows **custom public key** broadcasting for secure device identification.

Protocol and Specific Highlights

1. **Wi-Fi Scanning & RSSI Heatmapping:** The module scans all available Wi-Fi channels (1–13) to detect nearby networks, displaying SSID, BSSID, channel, and signal strength (RSSI). It creates heatmaps that visually represent coverage strength and dead zones, helping optimize access point placement and network performance.
2. **Wardriving Mode with GPS:** By integrating GPS data, the module records the geographic locations of detected Wi-Fi networks. This enables detailed mapping of signal coverage areas and assists in infrastructure audits, coverage optimization, and identifying weak signal regions.
3. **Deep Packet Inspection & Beacon Frame Analysis:** These tools analyze Wi-Fi packets and management frames to understand protocol behavior, encryption standards, and network configurations. They assist in detecting vulnerabilities, misconfigurations, and unusual network activity for security assessments.
4. **Offensive Testing Features (Ethical Use Only):** Includes advanced features such as deauthentication attacks, beacon spam (Evil Twin creation), and rogue access point (Evil Portal).

simulation to demonstrate potential wireless exploits. These should only be performed in controlled, authorized environments for penetration testing or educational purposes.





Bluetooth LE (Low Energy)

- Bluetooth LE is a power-efficient wireless communication standard designed for short-range data transmission.
- The scan detects nearby BLE devices like *iPhone_Pro* (-52dBm), *Fitbit_X2* (-65dBm), and *AirPods* (-73dBm), indicating signal strength and proximity.
- **Advertisement Monitoring** allows capturing device broadcast packets for analysis.
- **Device Fingerprinting** identifies unique characteristics of each BLE device for tracking or security research.
- **GATT Service Enumeration** explores available Bluetooth services and characteristics.
- **Proximity Tracking** estimates the distance based on RSSI values.
- **iBeacon Detection** identifies location beacons used for indoor navigation.
- **Offensive Features (listed but disabled)** include *Advertisement Spoofing*, *Sour Apple attack (iOS)*, and *BLE Jamming*, which are generally restricted for ethical use in controlled testing environments.

Sub-GHz (CC1101)

- The Sub-GHz module operates across *433 MHz*, *868 MHz*, and *915 MHz* frequency bands.
- Used for low-power, long-range communications such as IoT sensors, garage doors, and remote controls.
- **Scanning and Signal Capture** detects and logs transmissions for analysis.
- **ASK/OOK/FSK Modulation** support enables communication with various signal types.
- **Remote Control Detection** identifies signals from key fobs and smart devices.
- **Garage Door Monitoring** checks for transmissions within those bands.
- The *active band* (433.92 MHz) indicates real-time signal activity.
- **Offensive Features (disabled)** like *Signal Replay Attacks*, *RF Jamming*, and *Rolling Code Capture* demonstrate security vulnerabilities in radio systems, used only for ethical pentesting or research purposes.
- Helps cybersecurity experts study radio protocol weaknesses in IoT and embedded devices.



Peripheral Capabilities

RFID / NFC (PN532)

13.56 MHz

```
graph LR; PN532[PN532 NFC Module] -.-> MIFARE[MIFARE Classic]; PN532 -.-> NTAG[NTAG / Ultralight];
```

DEFENSIVE

- Card UID scanning
- Sector enumeration
- Access logging
- NDEF data parsing

OFFENSIVE (LAB)

- Card cloning/emulation
- NDEF write operations
- Demo replay attacks

IR Transceiver

38 kHz Carrier

Timing:
9us load + data
562.5us pulse width

NEC Protocol Example

DEFENSIVE

- Remote signal capture
- Protocol decoding
- Timing analysis
- SD logging

OFFENSIVE (LAB)

- Signal replay
- Universal remote TX
- Custom command inject

NRF24L01+ (2.4 GHz)

SCANNING & MONITORING

- Channel hopping (0-125)
- Packet sniffing & analysis
- Address space enumeration
- Mesh network mapping

DUAL-USE FEATURES

- Packet replay for testing
- Custom protocol dev
- Wireless sensor mesh
- IoT device testing

RFID / NFC (PN532) – 13.56 MHz

- The **PN532** module is widely used for **RFID (Radio Frequency Identification)** and **NFC (Near Field Communication)** applications operating at 13.56 MHz. It supports interaction with cards and tags like **MIFARE Classic** and **NTAG/Ultralight**, commonly used in access systems, payment cards, and smart devices.
 - **Defensive capabilities** include:
 - **Card UID Scanning** – retrieves the unique identifier of RFID/NFC tags.
 - **Sector Enumeration** – explores memory sectors and permissions on smart cards.
 - **Access Logging** – records interactions between the reader and tags.
 - **NDEF Data Parsing** – decodes data stored in the NFC Data Exchange Format, used in smart posters and contactless transfers.
 - **Offensive/Lab features** (for research only) include:
 - **Card Cloning/Emulation** – replicating or emulating tag behavior for system testing.
 - **NDEF Write Operations** – modifying data structures for authorized experiments.
 - **Demo Replay Attacks** – demonstrating data replay vulnerabilities in insecure systems.
 - This module is integral for understanding **NFC communication protocols**, tag authentication, and secure data exchange.
-

IR Transceiver – 38 kHz Carrier

- The **Infrared (IR) Transceiver** enables wireless communication using infrared light at a **38 kHz carrier frequency**, typical in TV remotes, air conditioners, and other consumer devices.
- **Defensive capabilities** include:
 - **Remote Signal Capture** – records raw IR transmissions.
 - **Protocol Decoding** – interprets encoding formats such as NEC, RC5, or Sony protocols.
 - **Timing Analysis** – measures pulse width and lead timings to identify protocol structures (e.g., 9 ms lead + 562.5 μ s pulse width for NEC).
 - **SD Logging** – stores captured IR data for further analysis.
- **Offensive/Lab features** (demonstration-focused) include:
 - **Signal Replay** – retransmits captured signals to test IR receiver robustness.

- **Universal Remote TX** – functions as a programmable remote capable of transmitting multiple device codes.
 - **Custom Command Injection** – sends tailored command sequences for testing system responses.
 - This transceiver aids in studying **IR communication standards**, timing behavior, and reverse engineering of consumer remote systems in a controlled setting.
-

NRF24L01+ (2.4 GHz)

- The **NRF24L01+** is a 2.4 GHz transceiver module commonly used in **IoT (Internet of Things)**, wireless sensors, and low-power communication networks.
- **Scanning and Monitoring features** include:
 - **Channel Hopping (0–125)** – scans all 126 channels in the 2.4 GHz band.
 - **Packet Sniffing & Analysis** – captures raw packets for decoding and protocol study.
 - **Address Space Enumeration** – maps device addresses to identify active nodes.
 - **Mesh Network Mapping** – visualizes multi-node communications in IoT environments.
- **Dual-use features** provide advanced research capabilities:
 - **Packet Replay for Testing** – reproduces captured packets for debugging and validation.
 - **Custom Protocol Development** – supports user-defined communication structures.
 - **Wireless Sensor Mesh** – enables interconnected low-power node networks.
 - **IoT Device Testing** – examines reliability, interference, and data integrity in 2.4 GHz IoT systems.
- This module is essential for **wireless communication analysis, cybersecurity testing, and IoT research**, emphasizing ethical use in monitoring and securing radio-based communication systems.

4.3 Testing

Test ID	Test Name	Objective	Test Steps	Expected Result
T-01	Wi-Fi Network Scan	Verify that OuroMini can detect nearby Wi-Fi networks	Power on device → Access web interface → Start Wi-Fi scan	List of available SSIDs displayed with RSSI, MAC address, and encryption type
T-02	BLE Advertisement Detection	Ensure BLE scanning functions correctly	Enable BLE scan mode → Observe nearby BLE devices	BLE advertisements displayed with device name, UUID, and signal strength
T-03	Promiscuous Packet Capture	Check if device captures raw Wi-Fi frames	Start packet capture in dashboard → Observe packet list	Packets captured and displayed with correct metadata
T-04	WPA/WPA2 Handshake Capture	Validate detection of WPA/WPA2 4-way handshake	Connect a device to a target network during scan	Handshake detected and saved as .pcap file
T-05	Web Dashboard Functionality	Verify real-time telemetry visualization	Connect to dashboard over LAN → Start monitoring	Data displayed live without significant lag
T-06	Data Logging to SD Card	Confirm SD card logging works properly	Insert SD card → Enable logging → Capture data	Data files correctly stored and accessible on SD card
T-07	OTA Firmware Update	Ensure firmware updates work wirelessly	Upload new firmware through web UI → Reboot device	Device runs updated firmware without data loss
T-08	Hardware Kill Switch	Test emergency stop feature	Activate hardware kill switch during operation	All wireless transmissions and captures immediately stop
T-09	Rate Limiting on Deauth Frames	Verify ethical frame injection control	Enable deauth mode → Monitor frame transmission rate	Frame rate limited to safe threshold
T-10	Web Interface Security	Ensure configuration access is encrypted	Attempt HTTP access → Attempt HTTPS access	HTTP access denied or redirected; HTTPS access successful and secure

Table of tests

5. Results

ESP32 Marauder Web Interface

Access Points

ESSID	Channel	Selected	RSSI	
Varma_4G	1	false	-35	false
Airtel_patt_4798	3	false	-94	false
Airtel_kati_3240	3	false	-94	false

Stations

MAC	RSSI	Selected
-----	------	----------

Probe Requests

SSID	Requests
------	----------

Fig 1-Web Interface

OuroMini Control Dashboard v1.4

• Live

ESP32-WROOM-32

Device Information

WiFi SSID
OuroMini

IP Address
192.168.4.1

Uptime
0h 5m 59s

Firmware
v1.4.0

DEVICE USAGE & ACCESS INFORMATION

- Commands are executed in the context of the **OuroMini** device and can be used to control its behavior, retrieve information, and manage settings.
- Connect to the OuroMini Wi-Fi Access Point "SSID: **OuroMini**, Password: **ASK ADMINISTRATOR**" to access this dashboard and manage your device.
- Use the terminal below to execute commands directly on the **OuroMini** device.
- Team: **OuroMini Firmware & Web Design Development Team**
- Version: **1.4**
- License: **AGPL-3.0**

OuroSpy Live Wi-Fi Data

Access Points, Stations, and Probe Requests

Access Points

SSID	Ch	Enc	RSSI	MAC
Varma_4G	1	WPA/WPA2	-41	B4:96:18:3D:E8:71
Thectan	3	WPA/WPA2	-91	20:0C:86:33:4A:48
JLG Wall-Mount A/C JB46	11	WPA2	-91	16:7F:67:D2:88:46
Airtel SUNNY POLA	11	WPA2	-94	58:07:F8:6A:62:FA

Fig 2- Wi-Fi Data listing

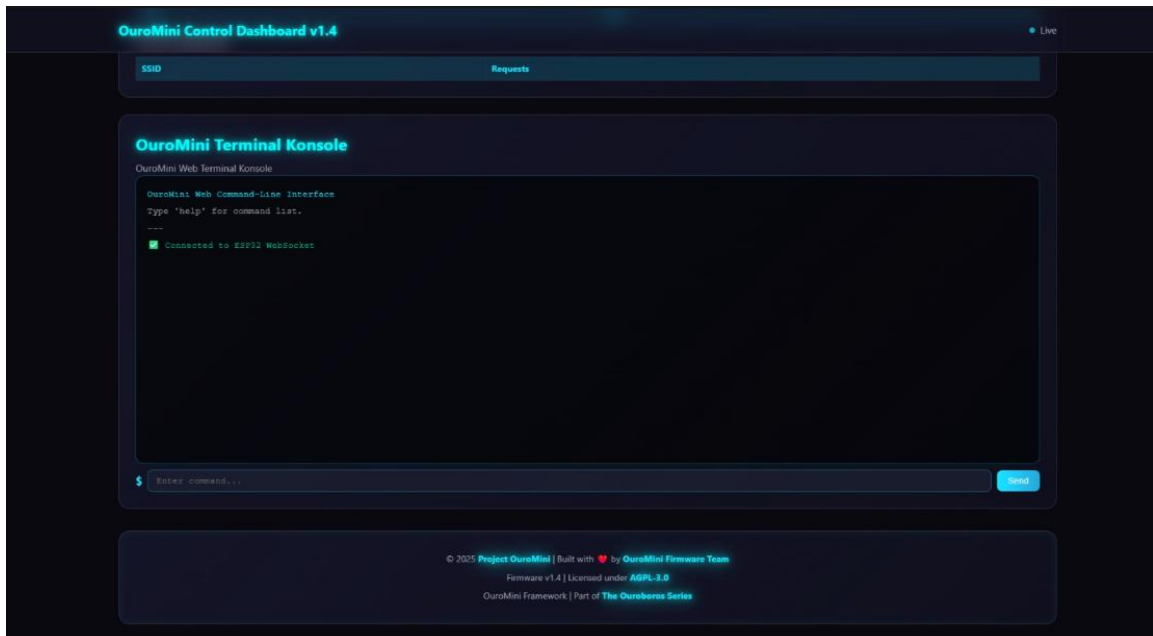


Fig 3- Terminal and dashboard

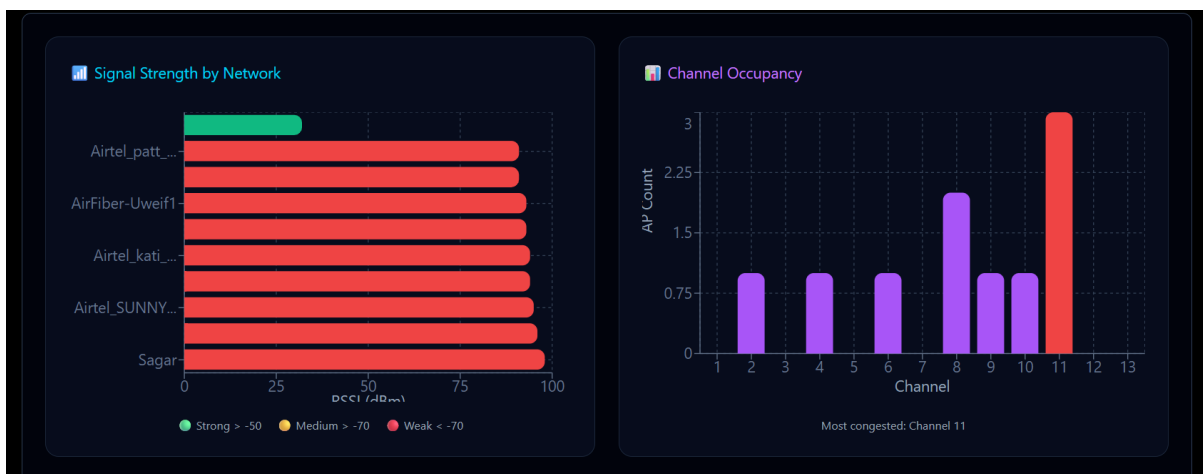


Fig 4- Network Overview and Signal Strength Distribution



Fig 5- Capture Summary

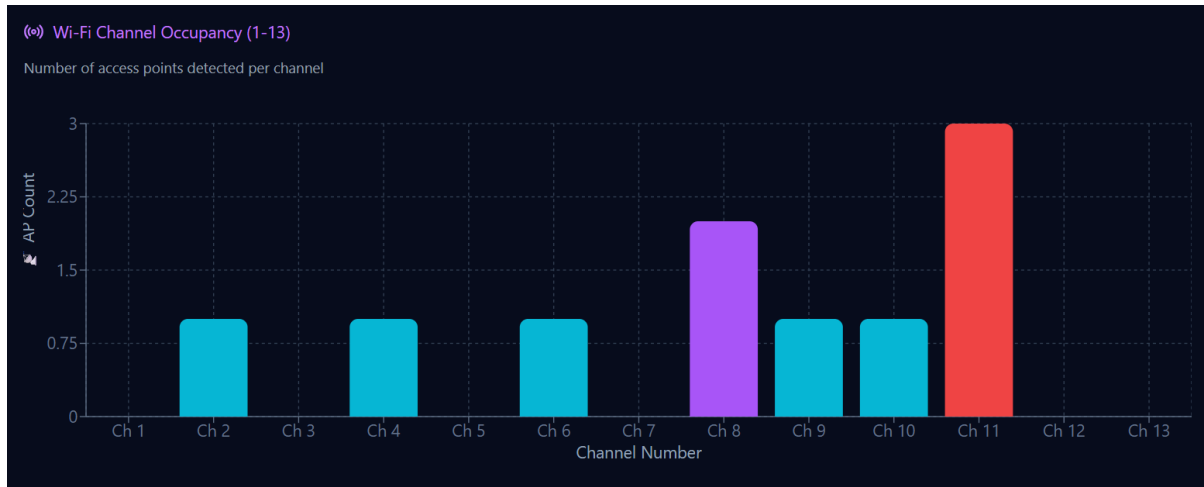


Fig 6- Channel Utilisation Analysis

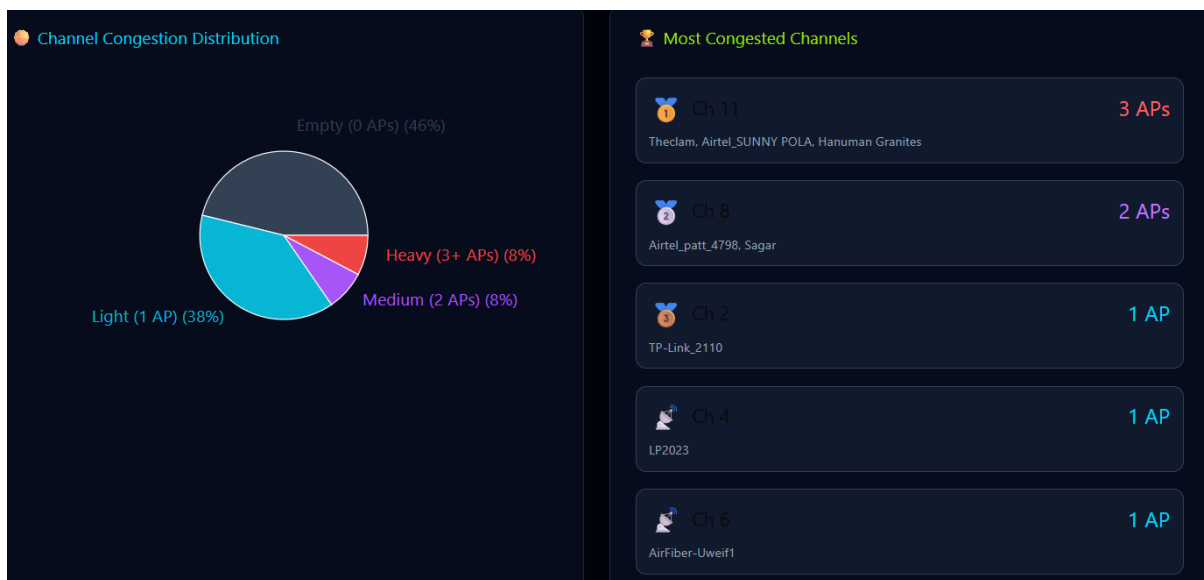


Fig 7- Channel Congestion Distribution



Fig 8- RSSI Signal Strength Heatmap

6. Conclusion

The Ouroboros Mini (OuroMini) project successfully demonstrates the transformation of an ESP32-WROOM32 module into a compact, headless, and feature-rich wireless monitoring and security research platform. Designed with an emphasis on portability, modularity, and ethical operation, OuroMini achieves its primary objectives through a robust firmware and hardware architecture that integrates both Wi-Fi and Bluetooth Low Energy (BLE) scanning capabilities.

The system supports promiscuous-mode packet capture, WPA/WPA2 four-way handshake detection and export, BLE advertisement monitoring, and real-time telemetry visualization via an onboard web dashboard accessible over a secure local network. Complementary features, including LittleFS and SD-based data logging, over-the-air (OTA) firmware updates, and a configurable web interface, extend operational flexibility and simplify field deployment. Safety mechanisms such as rate-limiting of probe and deauthentication frames, a hardware kill-switch, and encrypted configuration access ensure responsible and compliant usage within authorized environments.

OuroMini's architecture also emphasizes educational and demonstrative utility, enabling controlled offensive simulations to illustrate common wireless vulnerabilities—such as weak encryption, unsecured access points, and misconfigured BLE devices—without causing harm or disruption to production networks. Through these capabilities, the platform reinforces best practices in defensive network security, traffic analysis, and protocol forensics.

Ultimately, OuroMini serves as a compact, low-cost, and highly adaptable instrument for students, researchers, and security professionals seeking to explore wireless security concepts in a transparent and ethical manner. Its success validates the potential of embedded microcontrollers as powerful tools for modern cybersecurity education and research.

7.Future Scope

Although the current implementation is functional and robust for rapid prototyping, several enhancements can be considered for future development:

- **5 GHz Wi-Fi and Sub-GHz protocol support** using advanced chipsets or external modules
- **Enhanced BLE capabilities**, including authenticated pairing tests and improved MITM demonstrations
- **Machine learning–based anomaly detection** for real-time threat identification
- **Improved UI and analytics** including heatmaps, automated attack simulations, and dashboards
- **Hardware expansion** such as built-in IR modules, screen integration, and increased memory/storage
- **Modular firmware upgrades** supporting plug-in modules for NFC/RFID, LoRa, and SDR interfacing
- **Battery-powered mobile version** for wardriving and field research
- **Security hardening** including role-based access control for the web interface

These improvements can elevate OuroMini from a demonstration-oriented prototype into a more advanced and scalable wireless security platform suitable for extended academic and professional use.

References

1. Espressif Systems — *ESP32 Technical Reference Manual*.
<https://www.espressif.com/en/support/documents/technical-documents>
2. Espressif Systems — *ESP-IDF Programming Guide*.
<https://docs.espressif.com/projects/esp-idf/en/latest/>
3. Arduino Documentation — *ESP32 Board Support & Libraries*.
<https://docs.arduino.cc/hardware/esp32>
4. IEEE — *Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*.
IEEE Std 802.11™
5. Bluetooth SIG — *Bluetooth Low Energy Core Specifications*.
<https://www.bluetooth.com/specifications/>
6. PlatformIO — *Embedded Development Documentation*.
<https://docs.platformio.org/>
7. Wireshark Documentation — *Packet Capture Analysis Guide*.
<https://www.wireshark.org/docs/>
8. OWASP — *Wireless Security Testing Guidelines*.
<https://owasp.org/www-project-internet-of-things/>
9. LittleFS Documentation — *Lightweight Flash Filesystem*.
<https://github.com/ARMmbed/littlefs>
10. Rui Santos, Random Nerd Tutorials — *ESP32 Tutorials & Web Server Examples*.
<https://randomnerdtutorials.com/>

Appendix

- 1) GitHub Repository- <https://github.com/PardhuSreeRushiVarma20060119/Project-Ouroboros/tree/main/OuroMini>
- 2) Project Flyer Website- <https://stool-vast-77297611.figma.site/>
- 3) Report of detailed results- <https://appeal-ebook-83805802.figma.sit>

